Self-supervised Learning
Generative Models & Denoising

Hermann Blum

UNIVERSITÄT BONN

LAMARR INSTITUTE FOR MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Robot Perception and Learning Lab
rpl.uni-bonn.de

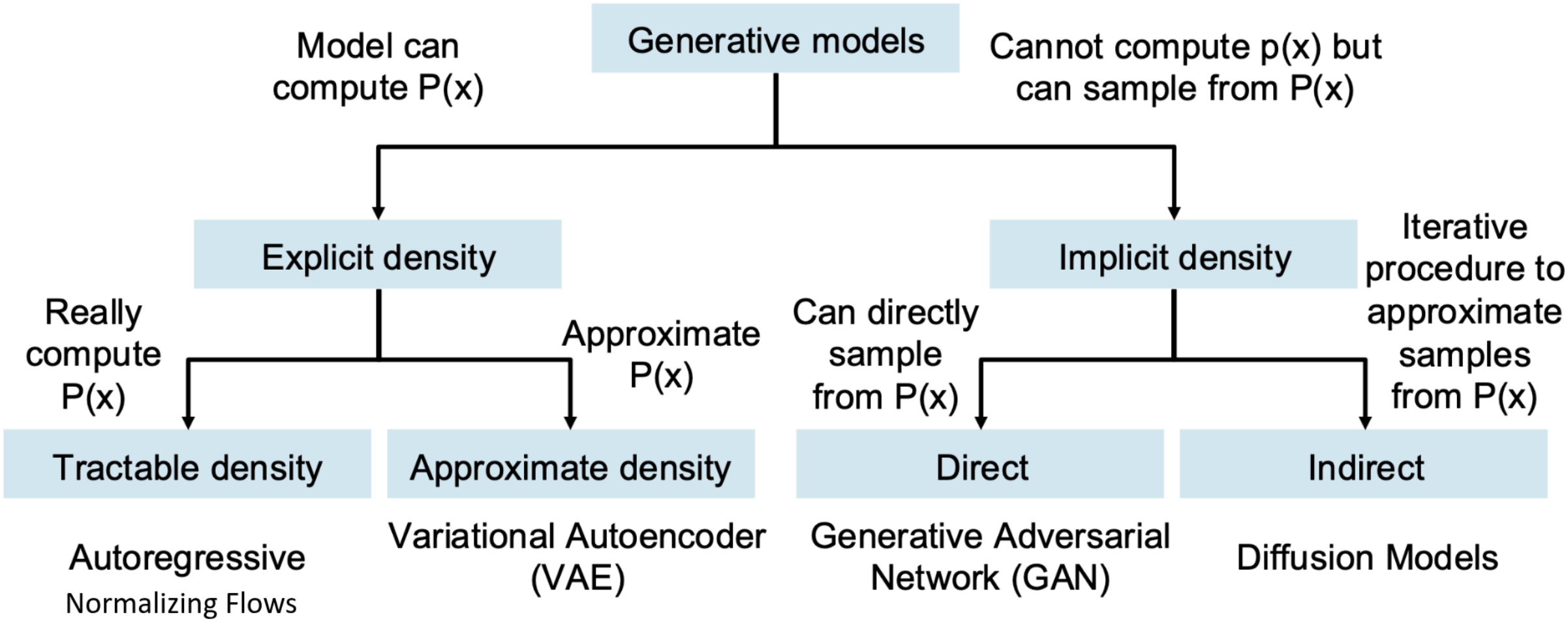
Important Reminder

Register for the exam on BASIS until June 21!

You can withdraw any registration until 7 days before the exam.

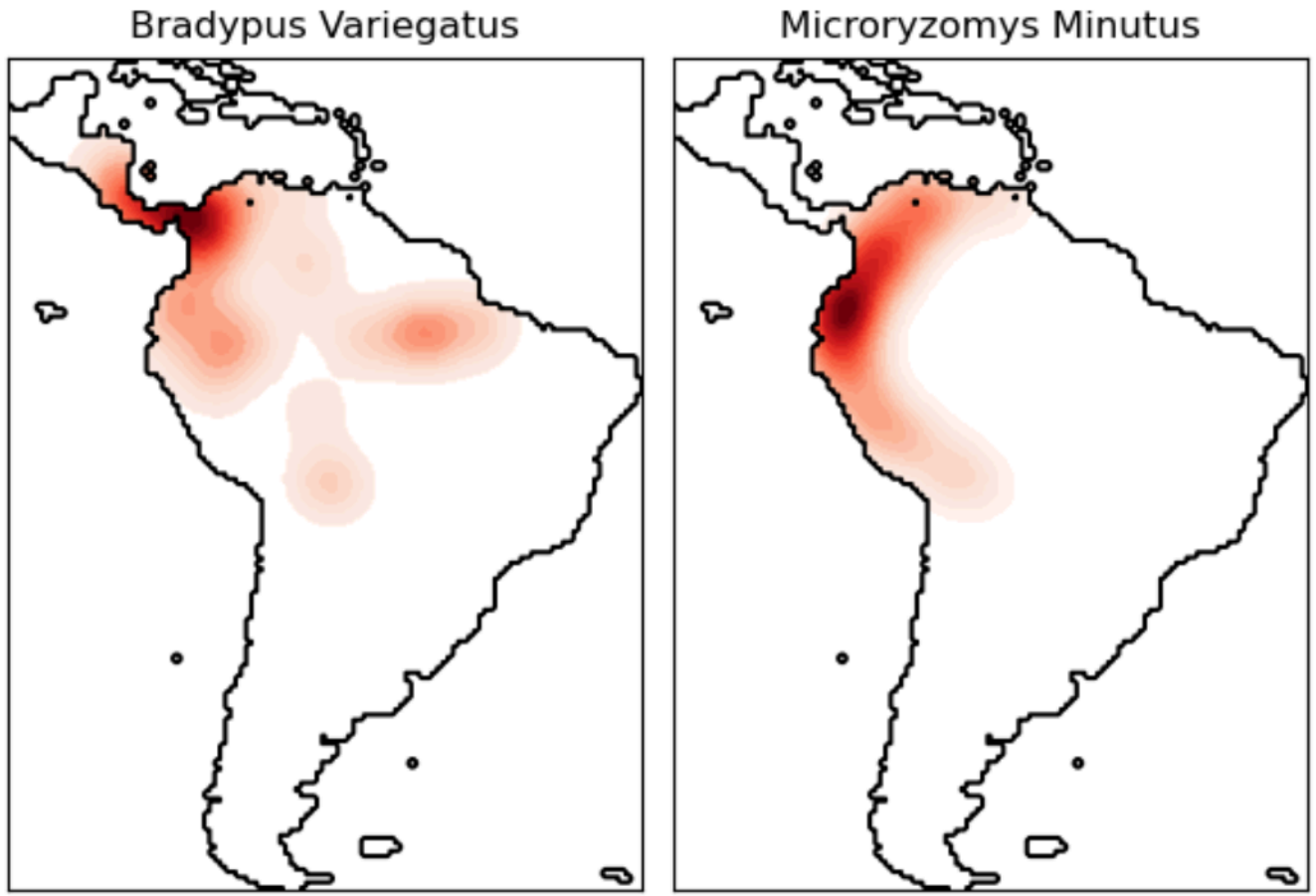
Taxonomy of Generative Models

Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017



slide by Li et al, Stanford CS231n

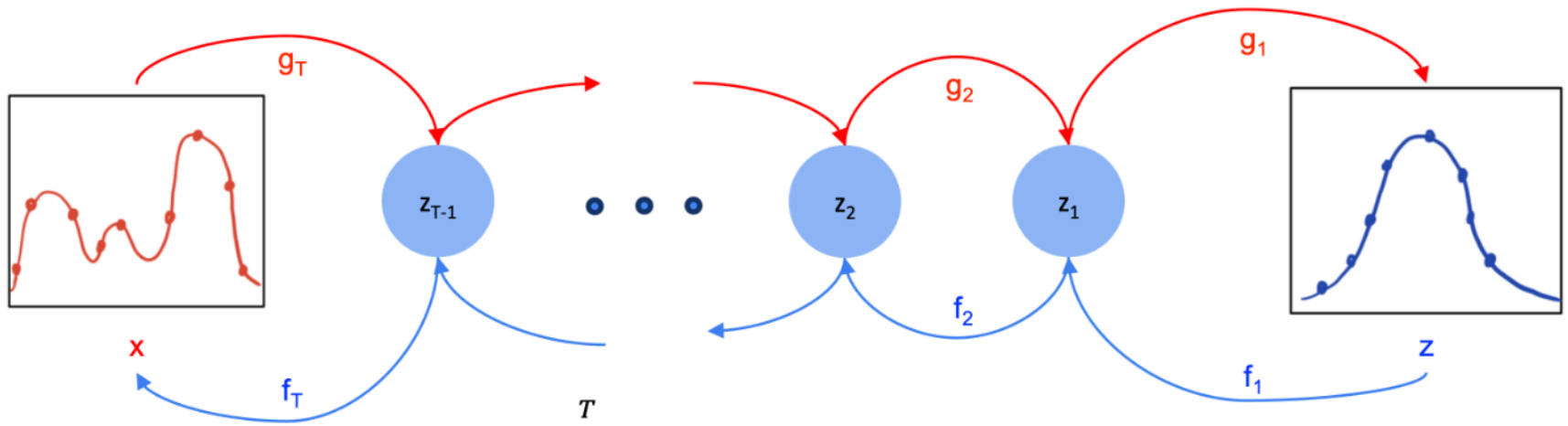
Kernel Density Estimation



Normalizing Flows

Composing many layers together:

$$z \sim p(z)$$
$$x = f_T \circ f_{T-1} \circ \cdots \circ f_1\,(z)$$



$$\log p_x(x) = \log p_z(z_0) + \sum_{t=1}^{T} \log\left|det.J_{z_t}\left(g(z_{t-1})\right)\right|, \qquad z_T = x,\ z_0 = z$$

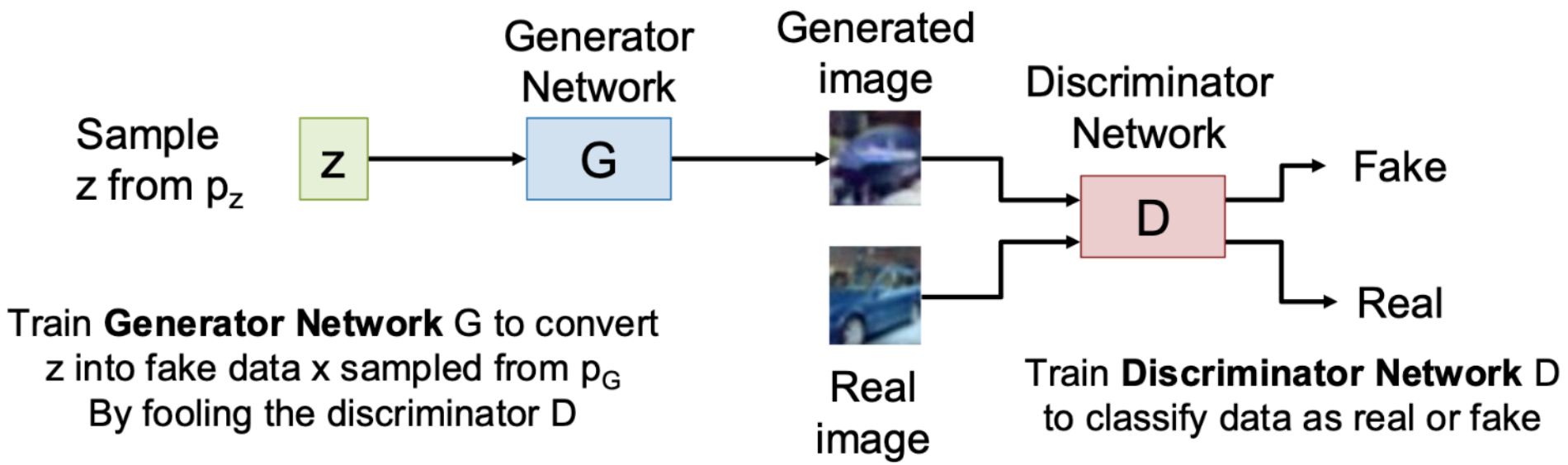
Inverse: $z = g_1 \circ g_2 \circ \cdots \circ g_T(x)$

slide by Abuzar Khan, CMU

Generative Adversarial Networks

Setup: Have data x_i drawn from distribution $p_{data}(x)$. Want to sample from p_{data}

Idea: Introduce a latent variable z with simple prior p(z) (e.g. unit Gaussian)
Sample $z \sim p(z)$ and pass to a **Generator Network** x = G(z)
Then x is a sample from the **Generator distribution** p_G. Want $p_G = p_{data}$!

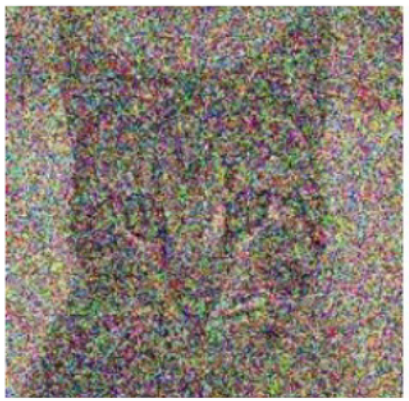


slide by Li et al, Stanford CS231n

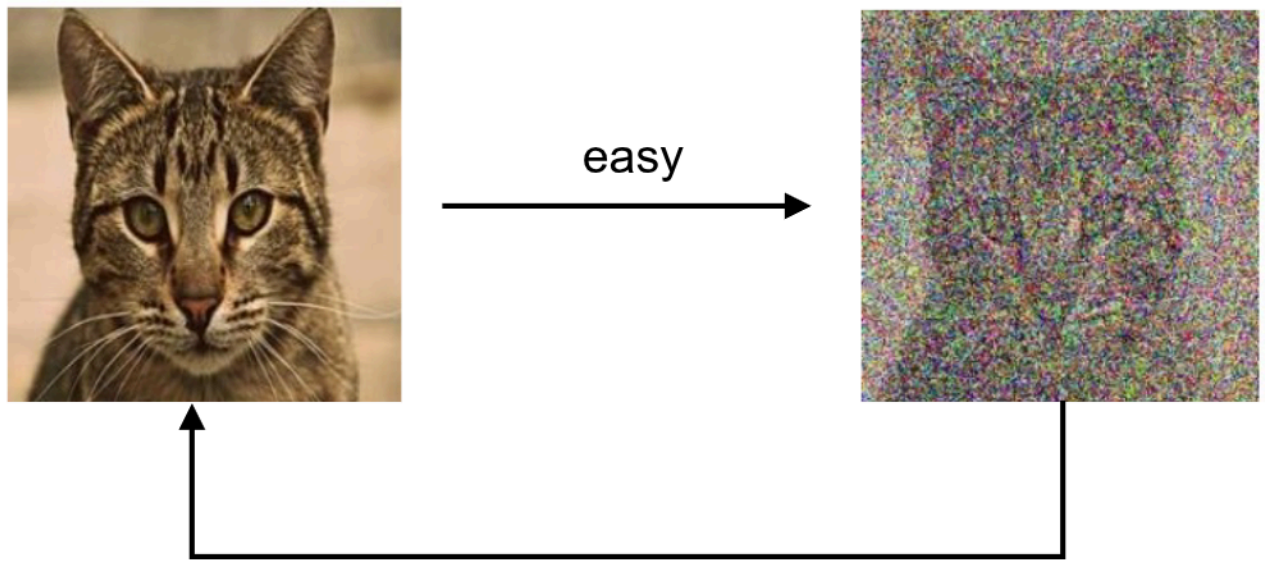
Denoising

Denoising

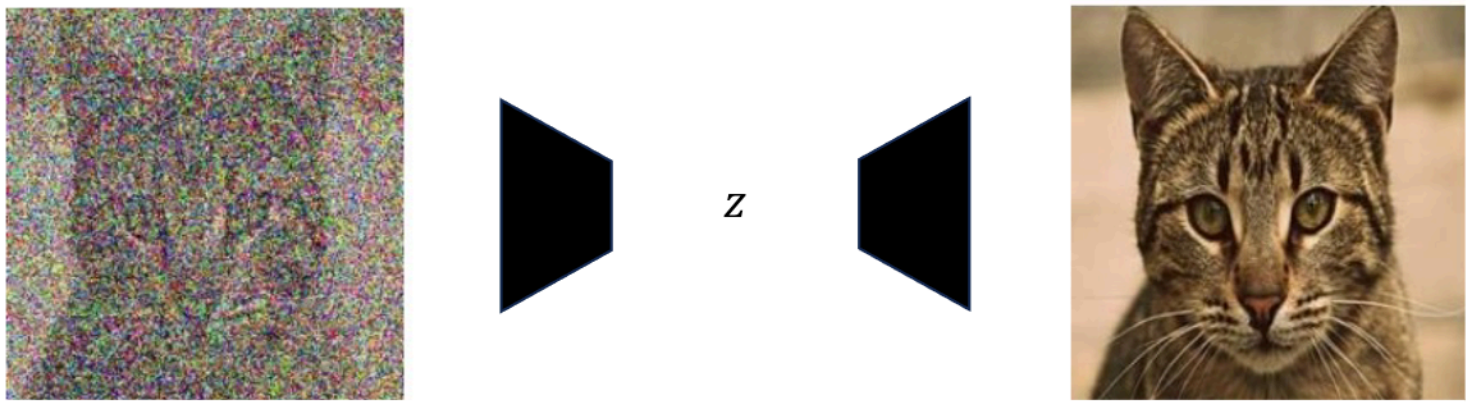
easy



Denoising

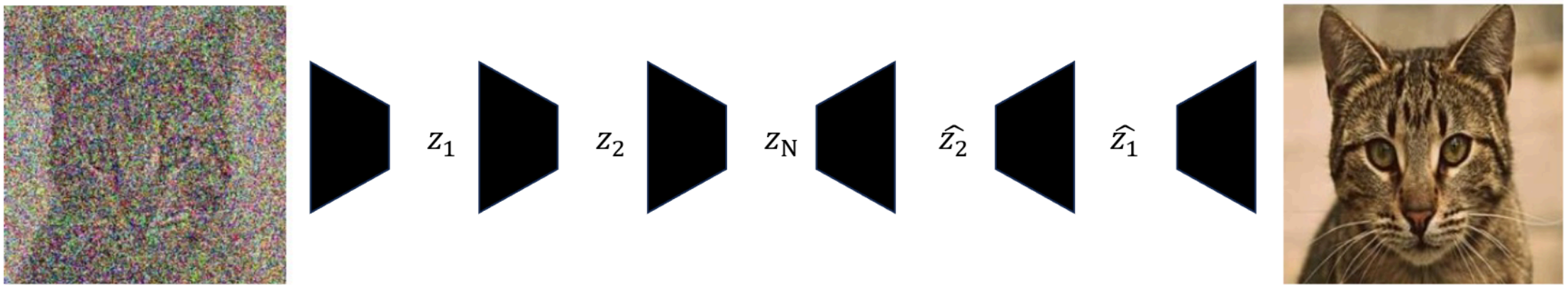


Denoising Autoencoder



- similar to masked autoencoder: has to learn abstract structure in the data for z, fill detail in only in decoder
- improves robustness compared to standard AE

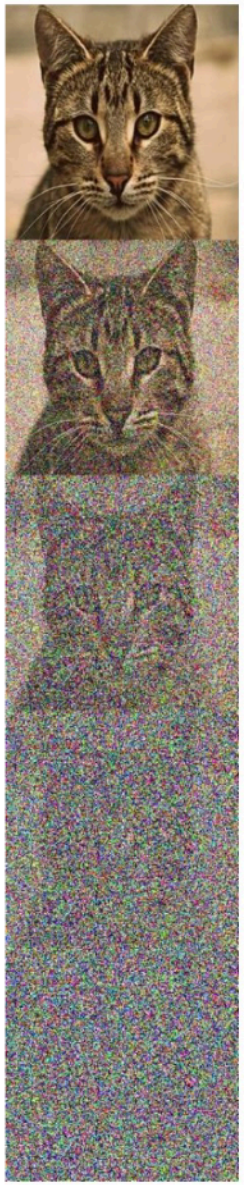
Stacked Denoising Autoencoder



- layerwise pretraining, apply noise to output from outer encoder → easier with noise than with masking
- common method for representation learning, yields abstract features

Diffusion Models

Core principal:
learn gradual denoising,
then apply this process many times.



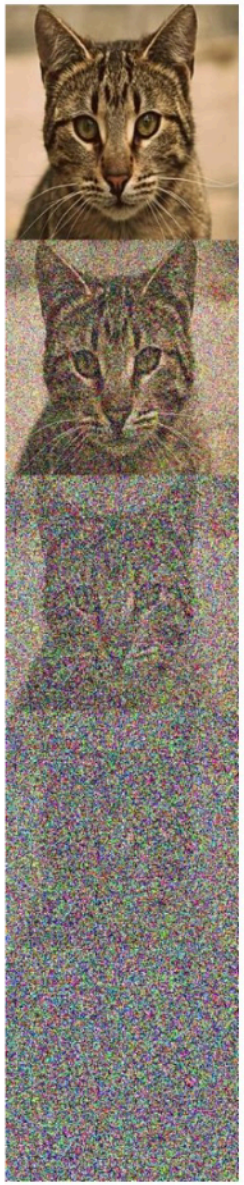
t = 0
No noise

t = 1
Full noise

slide by Li et al, Stanford CS231n

Diffusion Models

Pick a **noise distribution** $z \sim p_{noise}$



t = 0
No noise

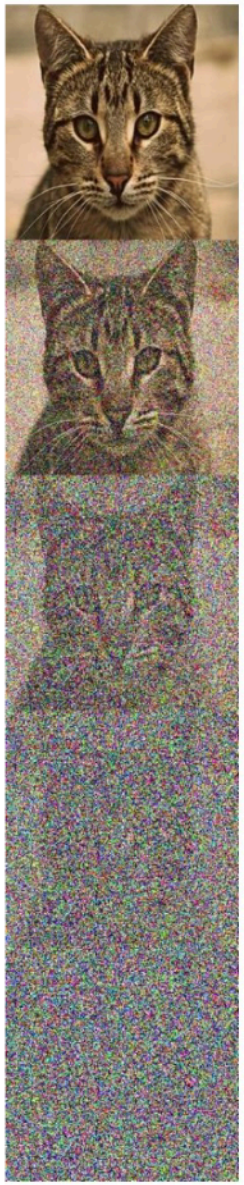
t = 1
Full noise

slide by Li et al, Stanford CS231n

Diffusion Models

Pick a **noise distribution** $z \sim p_{noise}$
(Usually unit Gaussian)

Consider data x corrupted under varying **noise levels** t to give noisy data x_t



t = 0
No noise

t = 1
Full noise

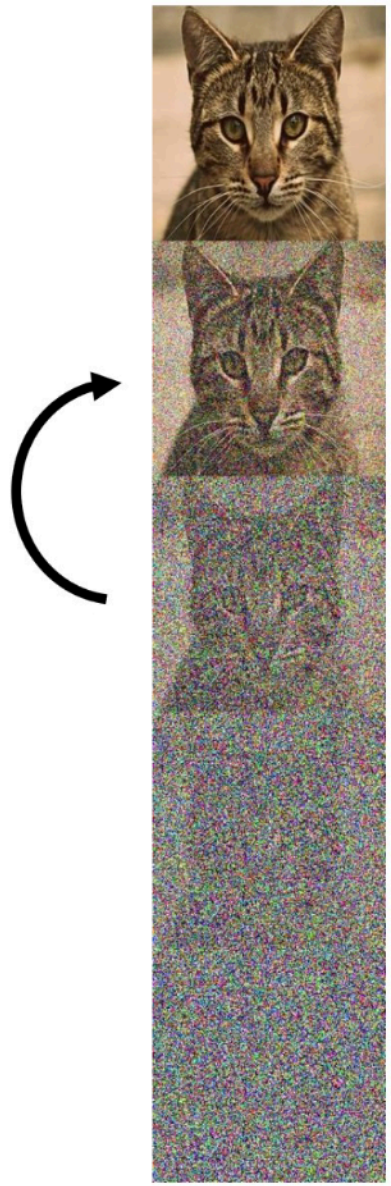
slide by Li et al, Stanford CS231n

Diffusion Models

Pick a **noise distribution** $z \sim p_{noise}$
(Usually unit Gaussian)

Consider data x corrupted under varying **noise levels** t to give noisy data x_t

Train a neural network to **remove a little bit of noise**: $f_\theta(x_t, t)$



t = 0
No noise

t = 1
Full noise

slide by Li et al, Stanford CS231n

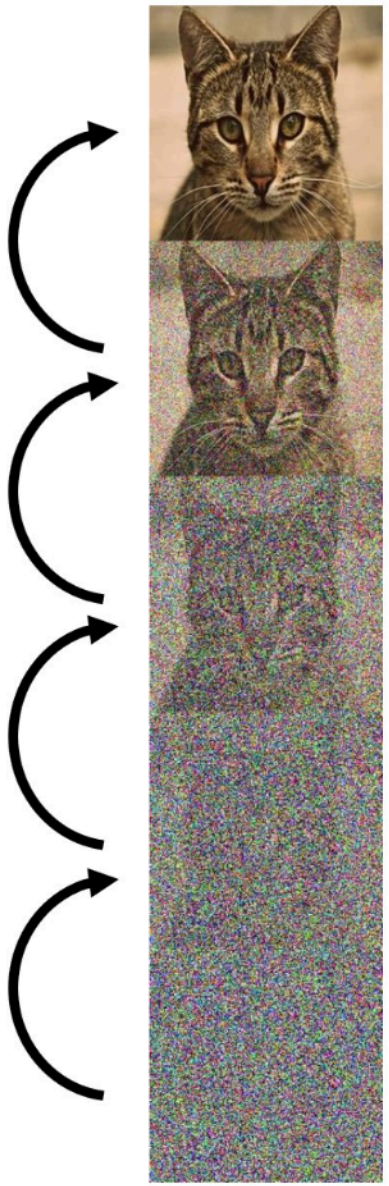
Diffusion Models

Pick a **noise distribution** $z \sim p_{noise}$
(Usually unit Gaussian)

Consider data x corrupted under varying **noise levels** t to give noisy data x_t

Train a neural network to **remove a little bit of noise**: $f_\theta(x_t, t)$

At inference time, sample $x_1 \sim p_{noise}$ and apply f_θ many times in sequence to generate a noiseless sample x_0



slide by Li et al, Stanford CS231n

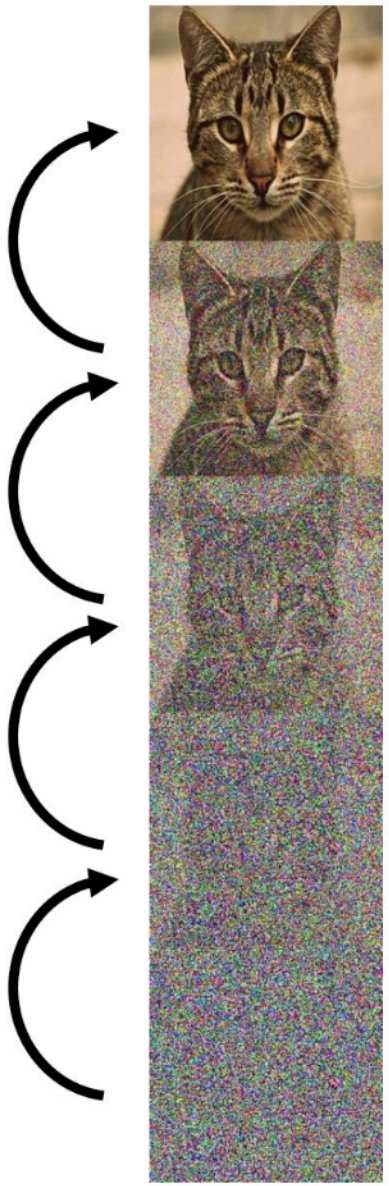
Diffusion Models

Pick a **noise distribution** $z \sim p_{noise}$
(Usually unit Gaussian)

Consider data x corrupted under varying **noise levels** t to give noisy data x_t

Train a neural network to **remove a little bit of noise**: $f_\theta(x_t, t)$

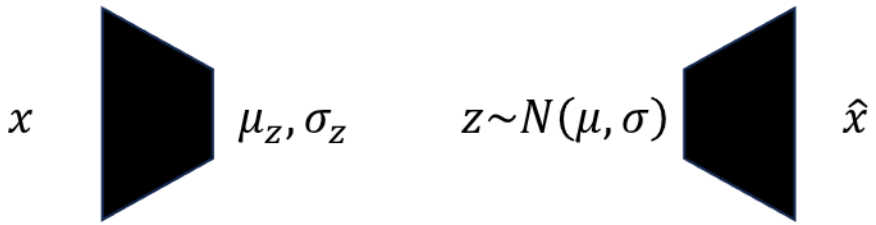
At inference time, sample $x_1 \sim p_{noise}$ and apply f_θ many times in sequence to generate a noiseless sample x_0



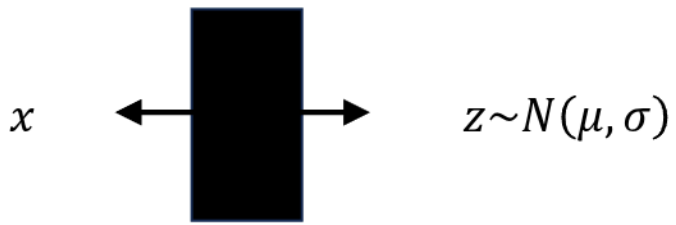
slide by Li et al, Stanford CS231n

Concept Comparison

Variational Autoencoder

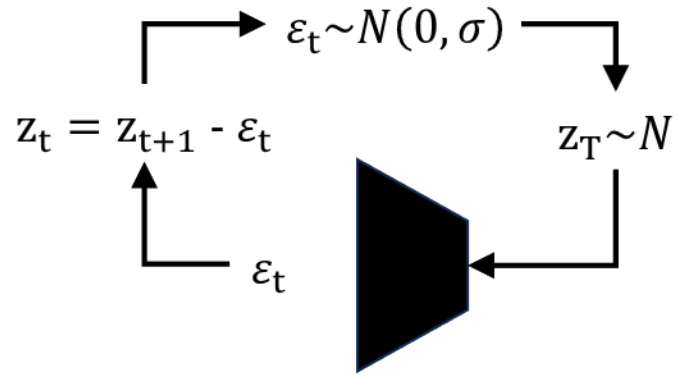


Normalizing Flows:



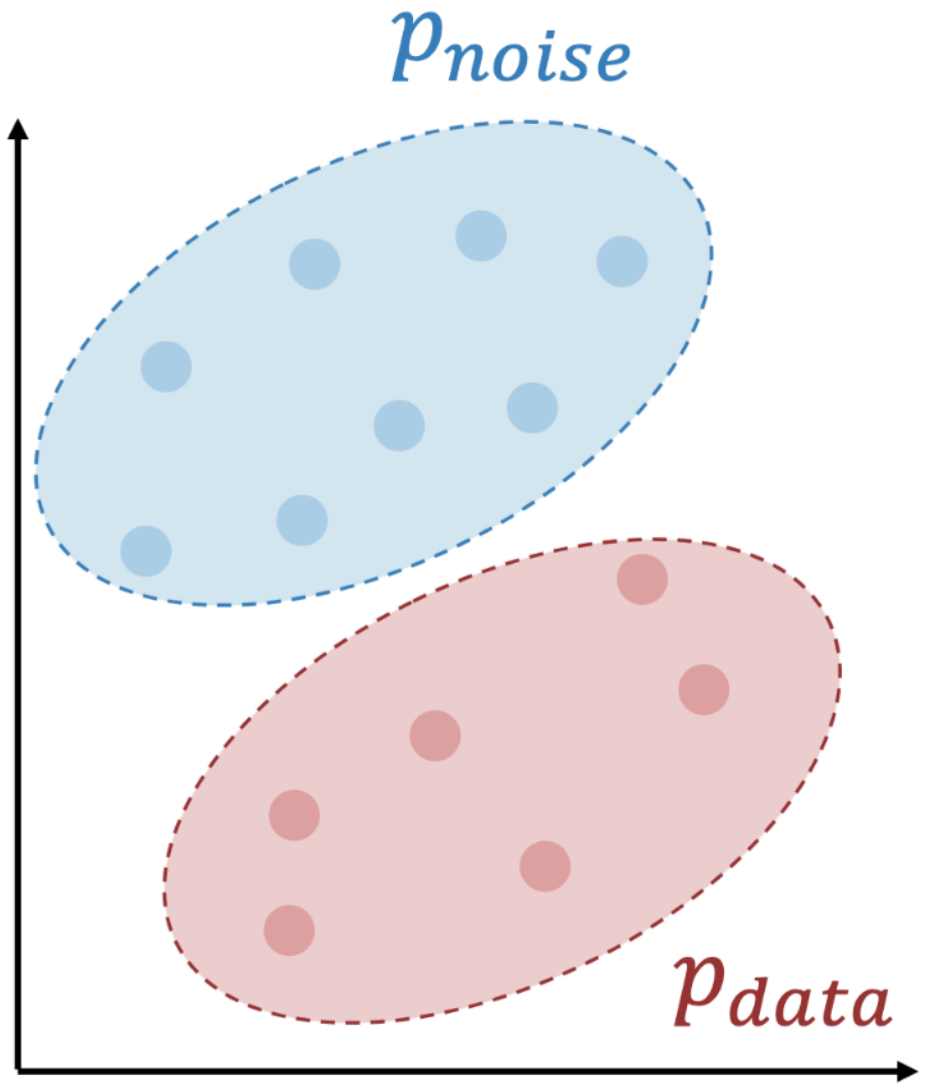
Diffusion Models:

$x = z_1 - \varepsilon_1$



Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

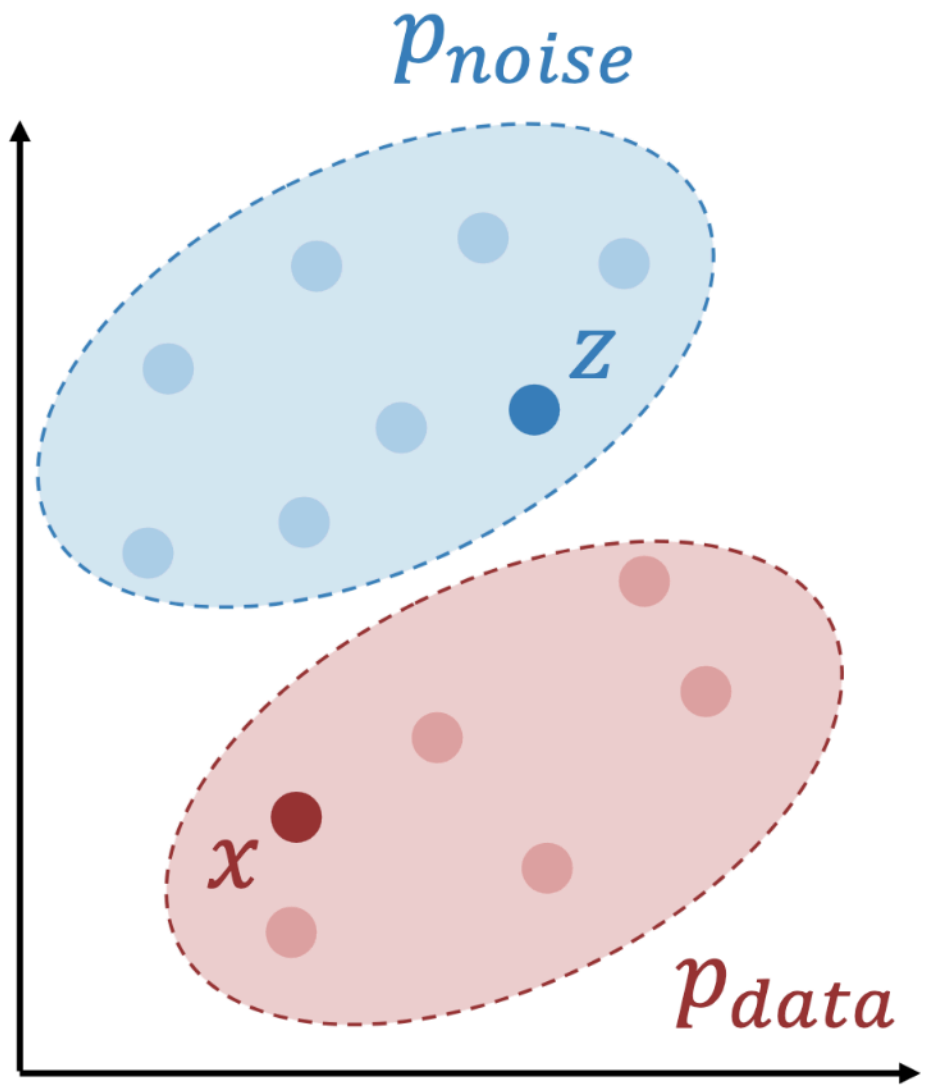
slide by Li et al, Stanford CS231n

Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}

On each training iteration, sample:

$z \sim p_{noise}$ $x \sim p_{data}$ $t \sim Uniform[0, 1]$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

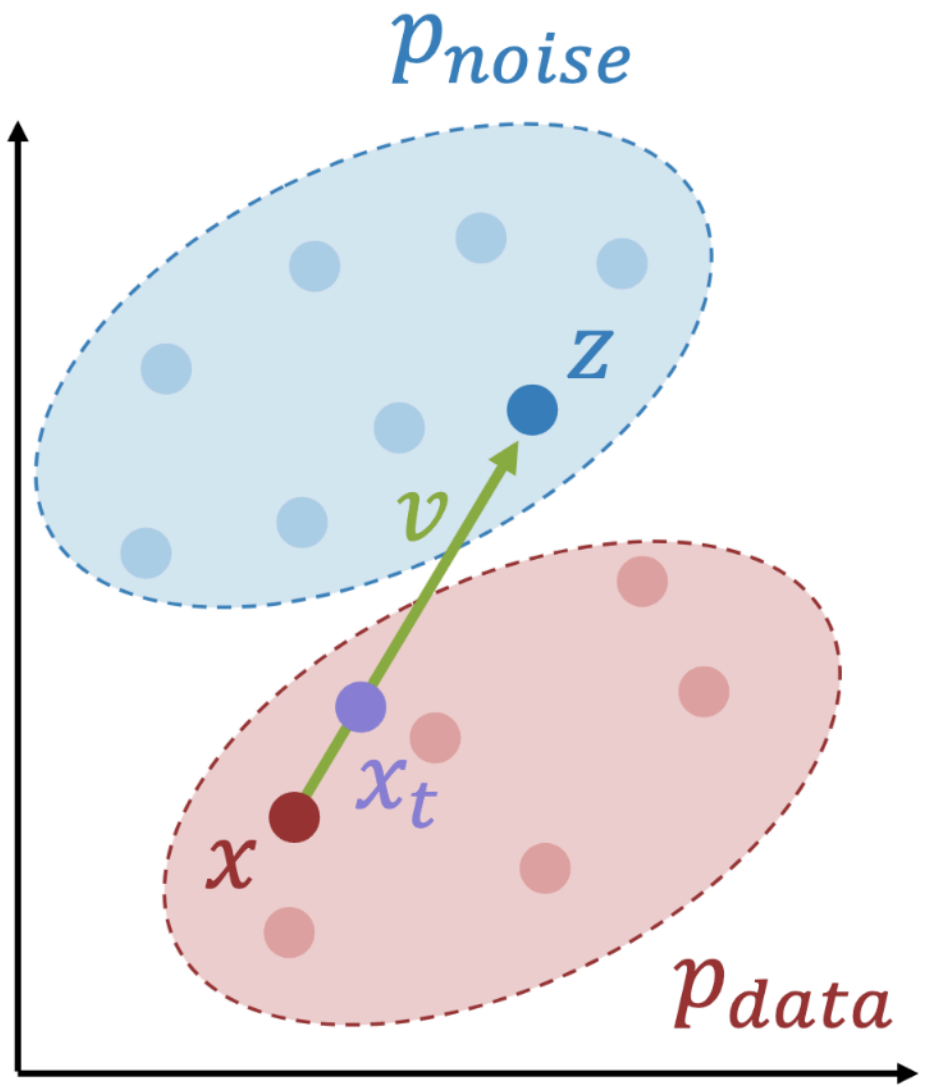
Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}

On each training iteration, sample:

$z \sim p_{noise}$ $x \sim p_{data}$ $t \sim Uniform[0, 1]$

Set $x_t = (1-t)x + tz$, $v = z - x$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Diffusion Models: Rectified Flow

Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}

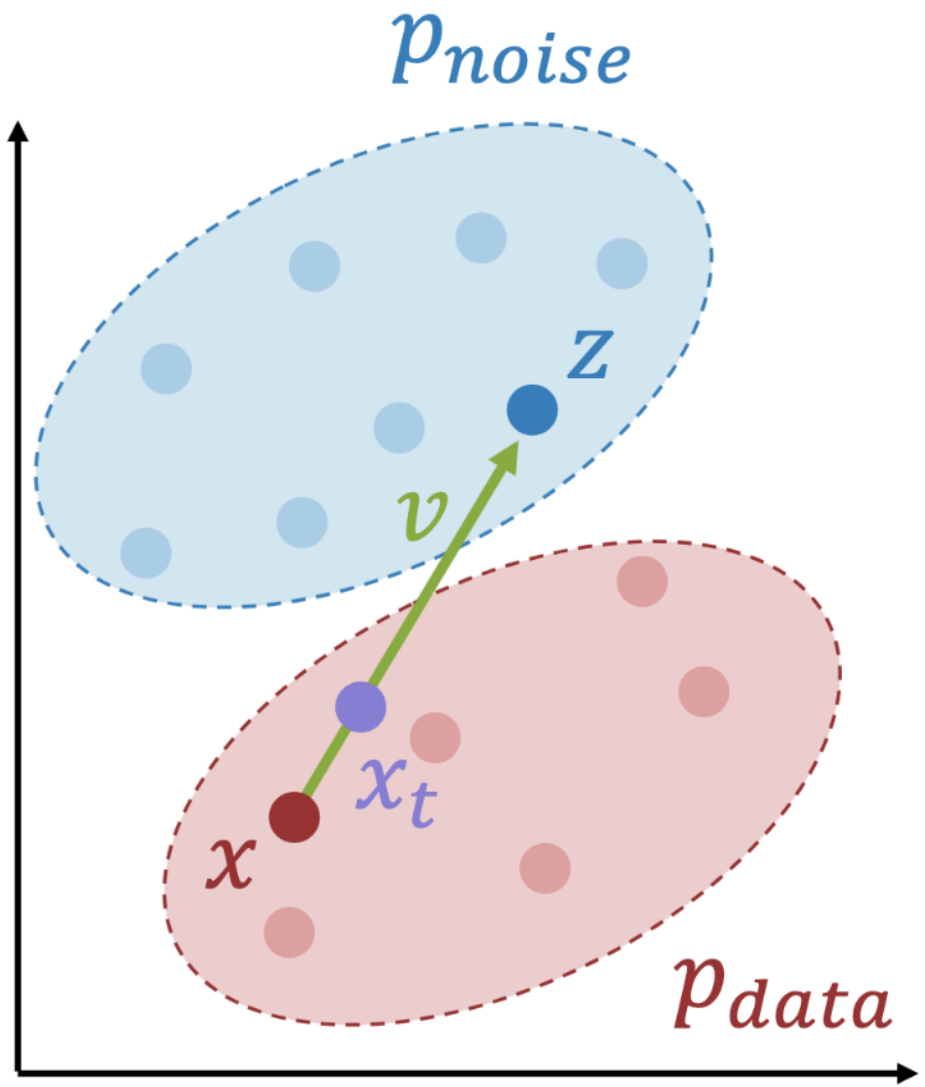
On each training iteration, sample:

$z \sim p_{noise}$ $x \sim p_{data}$ $t \sim Uniform[0, 1]$

Set $x_t = (1-t)x + tz$, $v = z - x$

Train a neural network to predict v:

$$L = \|f_\theta(x_t, t) - v\|_2^2$$

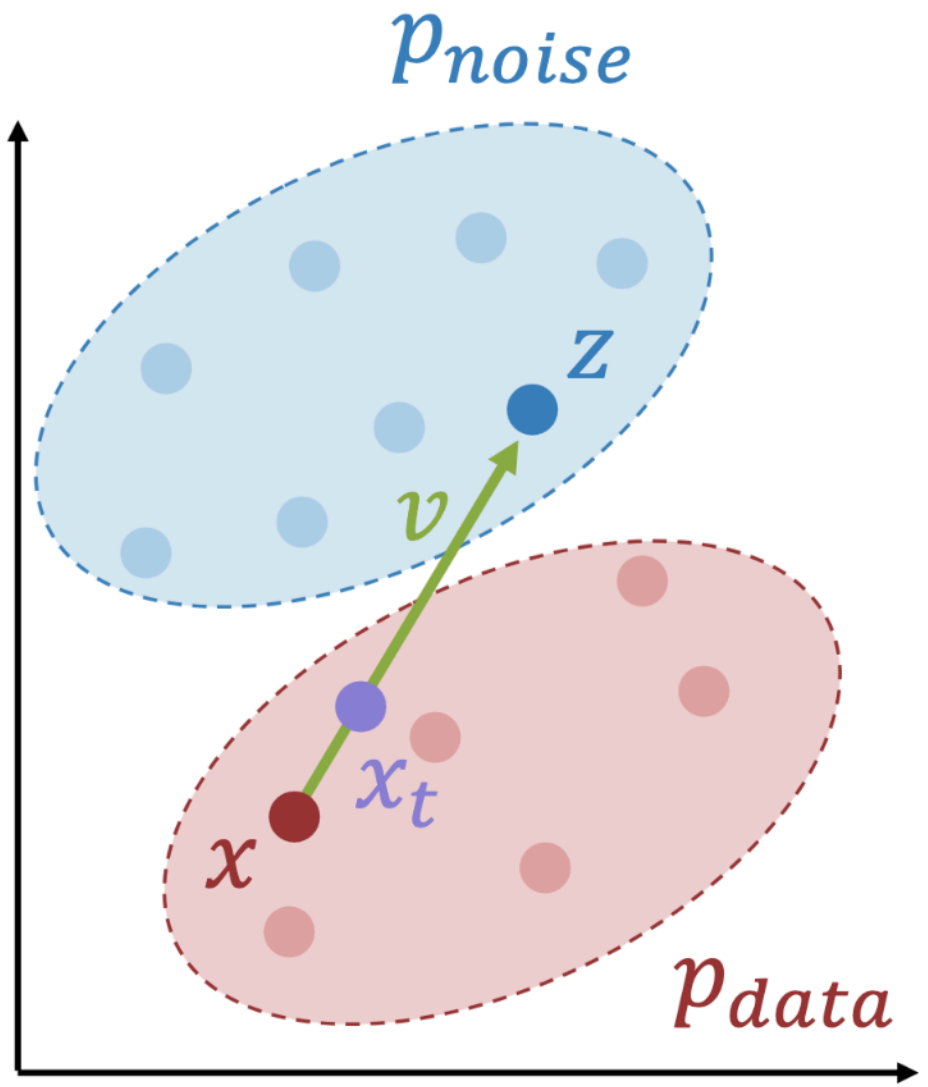


Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Diffusion Models: Rectified Flow

Core training loop is just
a few lines of code!

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

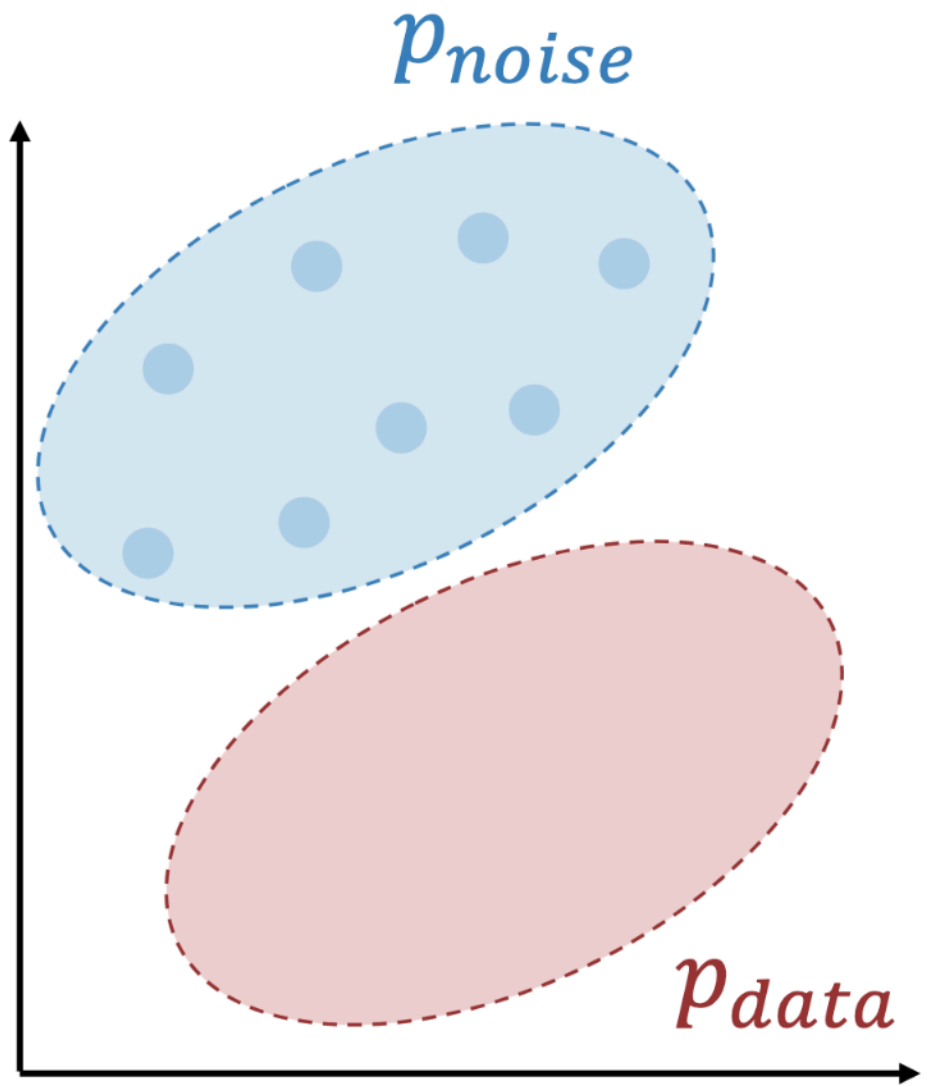


Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

Choose number of steps T (often T=50)



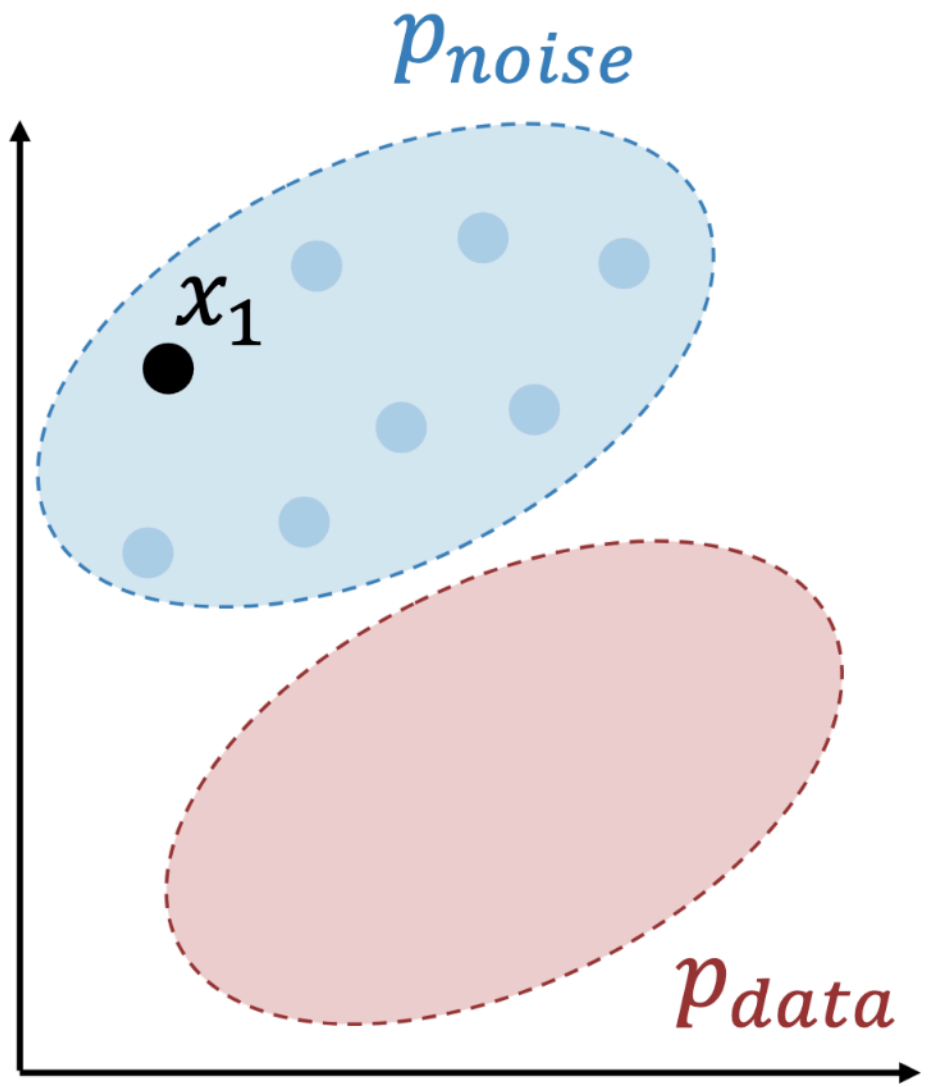
Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

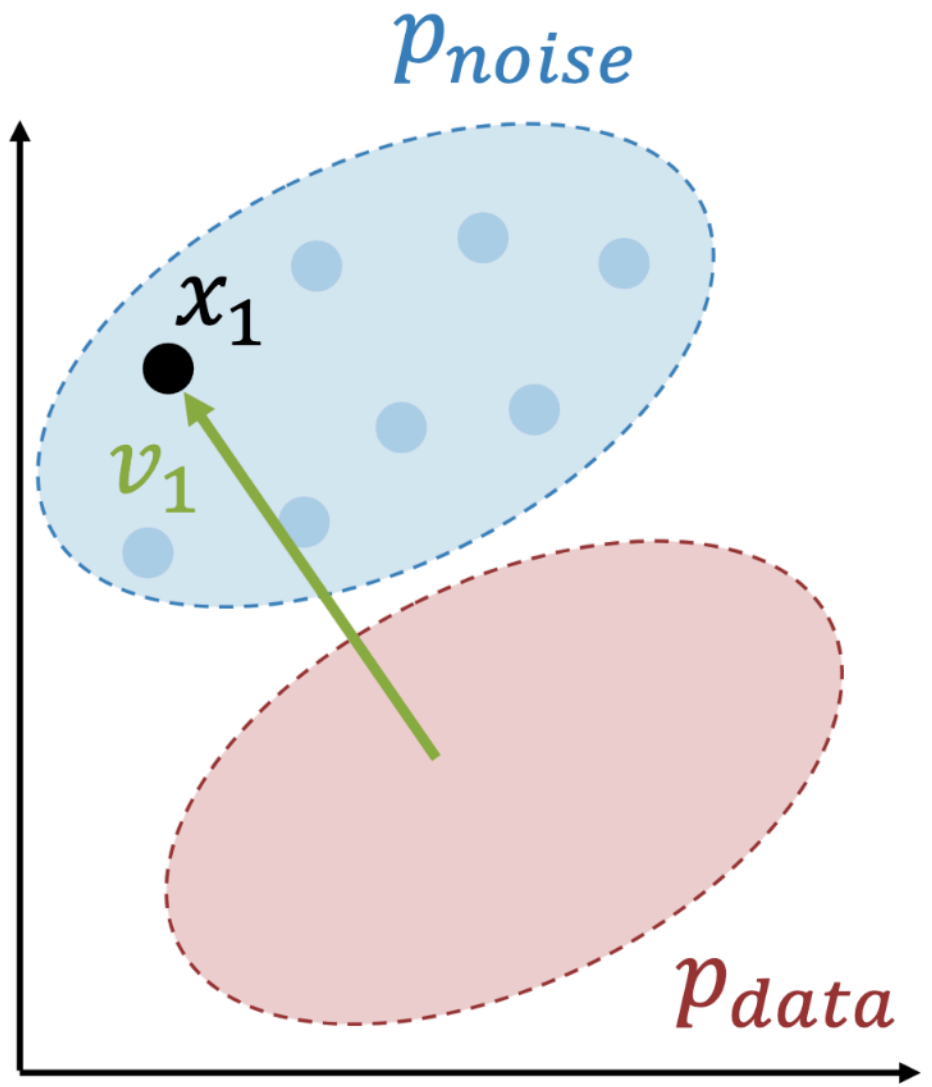
Rectified Flow: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Rectified Flow: Sampling

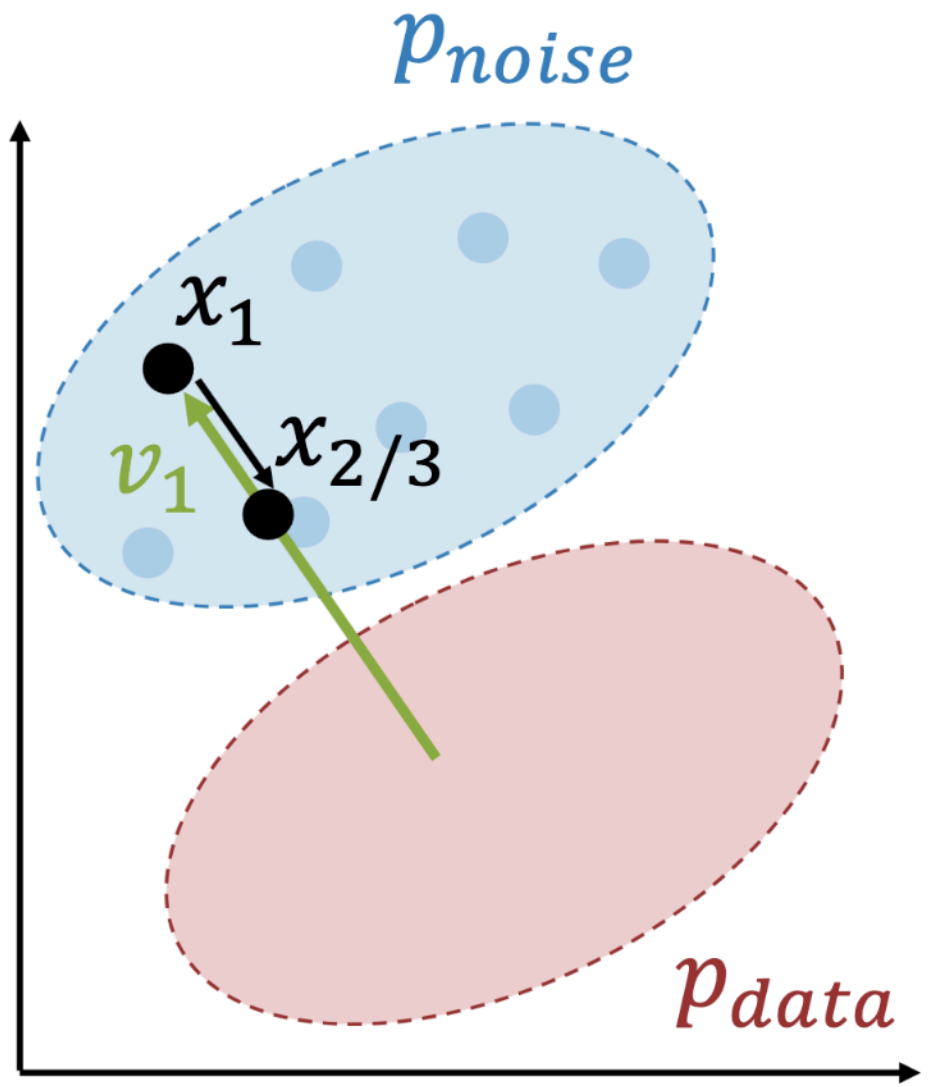
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

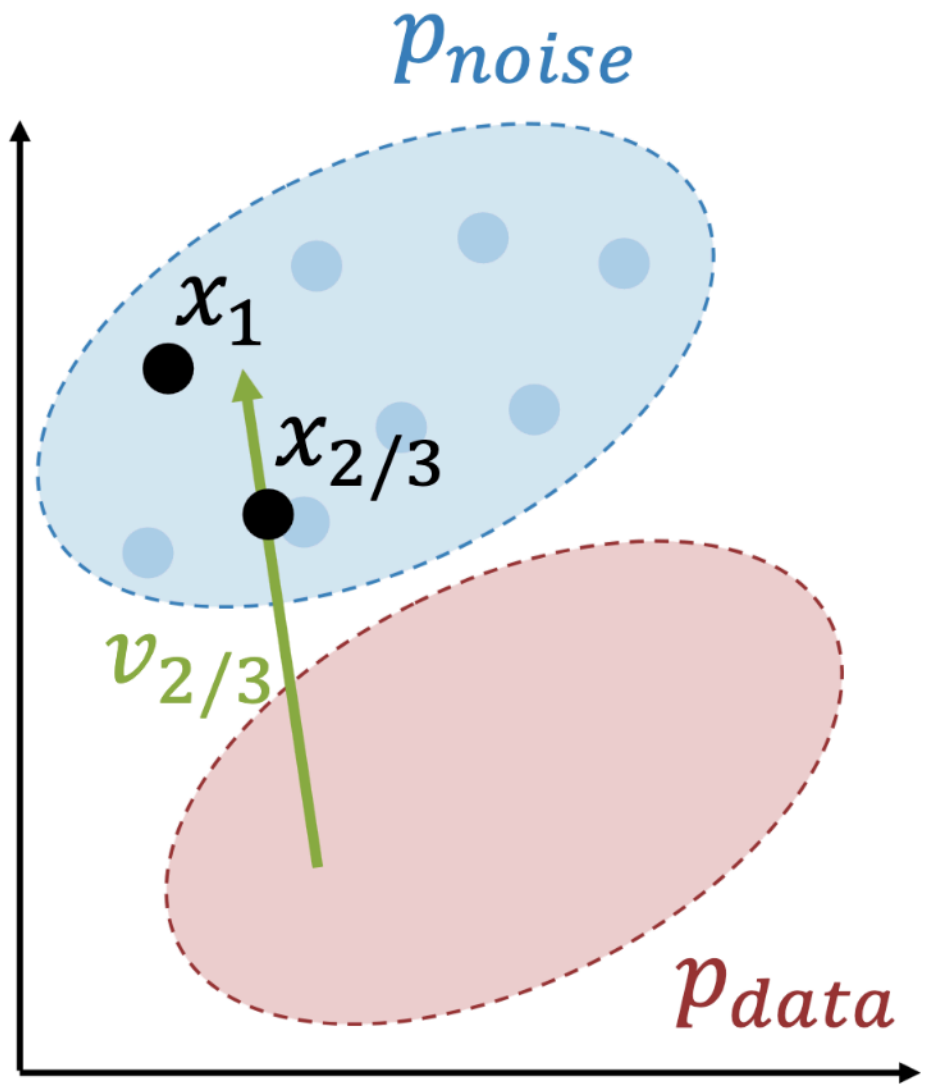
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

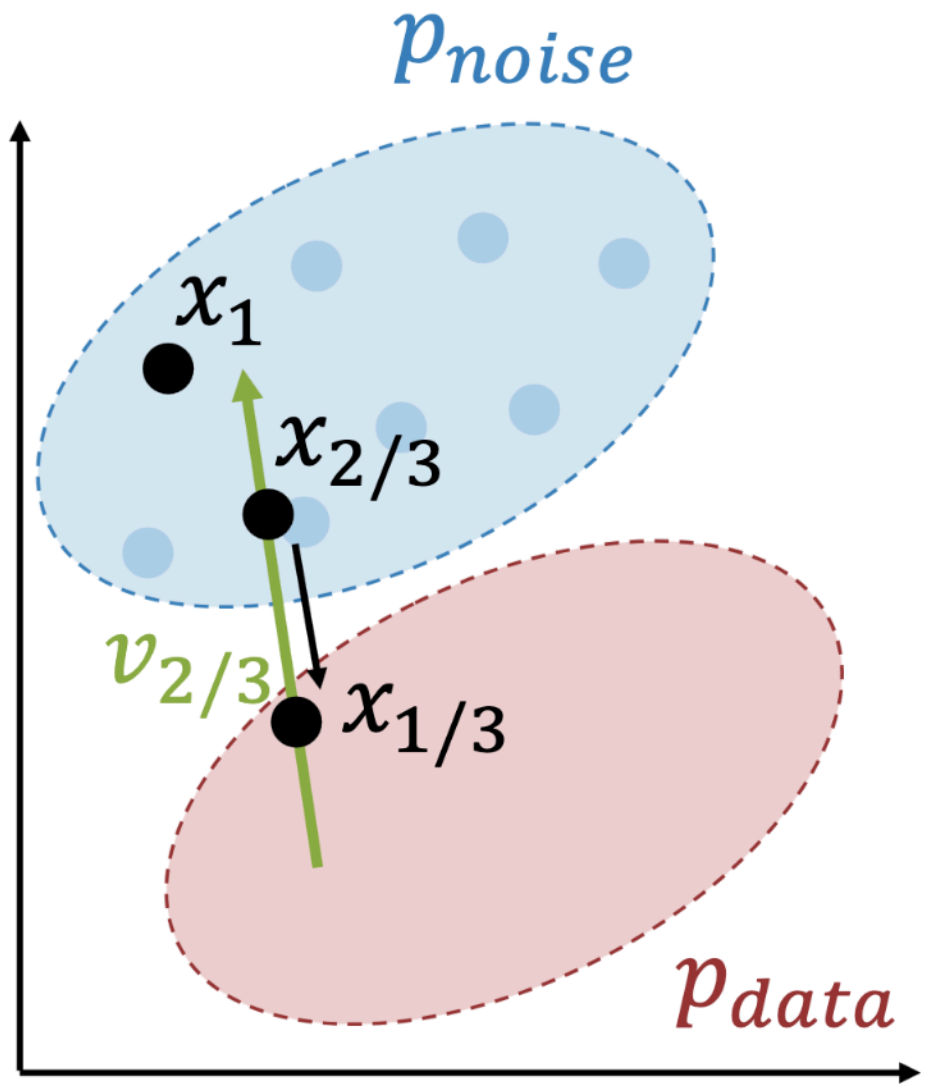
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

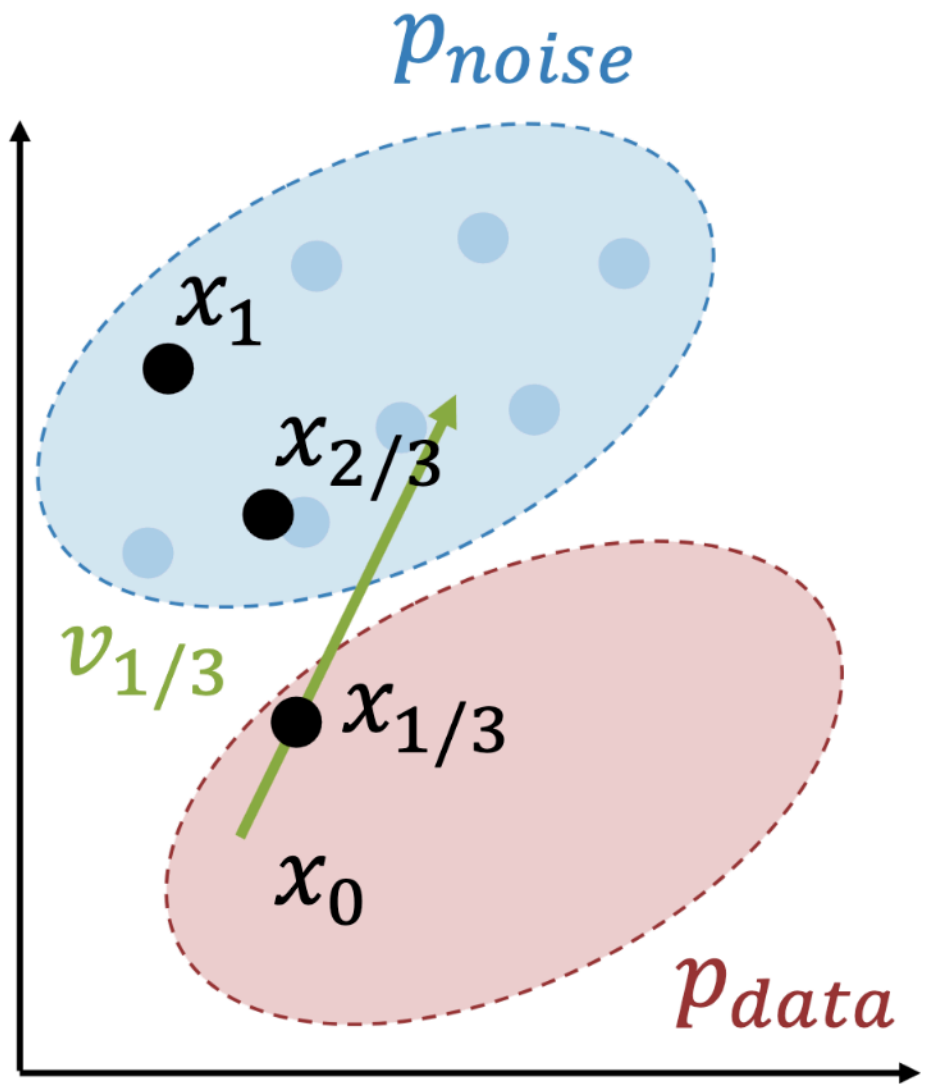
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

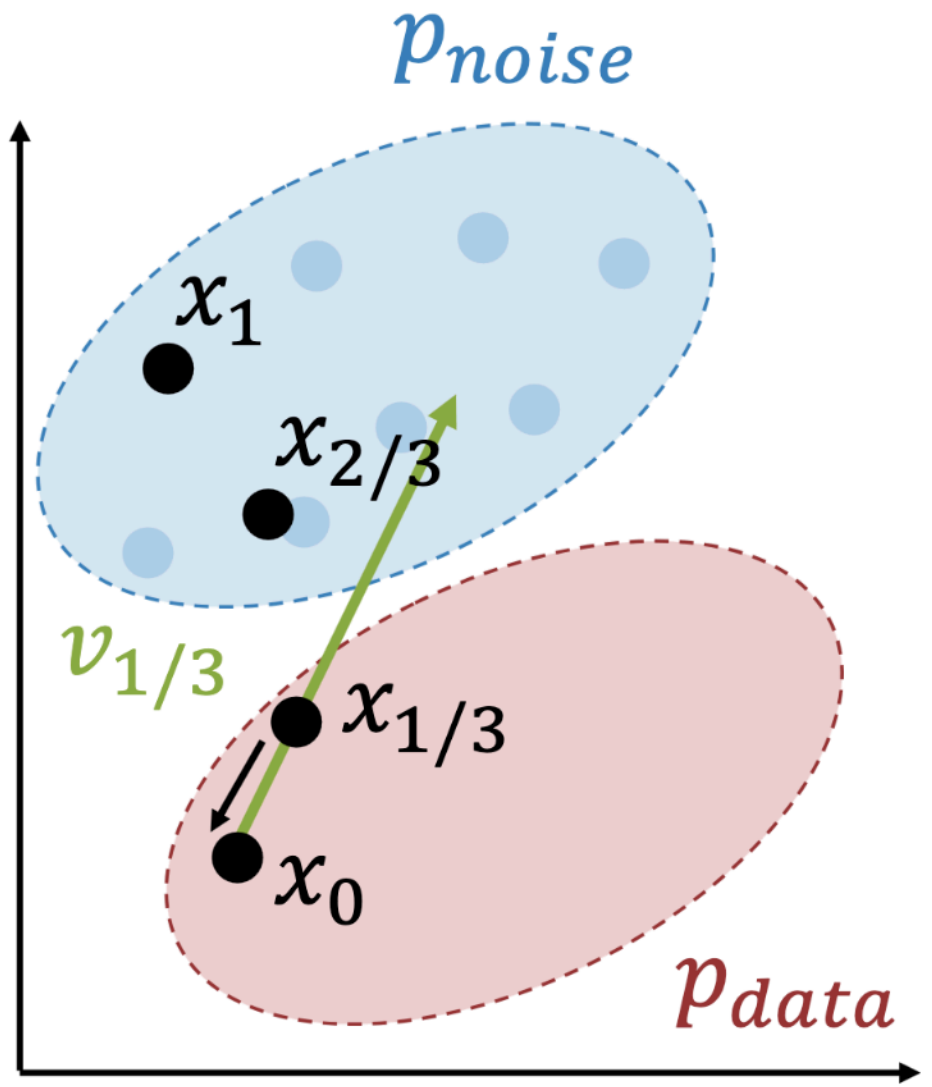
Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

Evaluate $v_t = f_\theta(x_t, t)$

Step $x = x - v_t/T$



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

Choose number of steps T (often T=50)

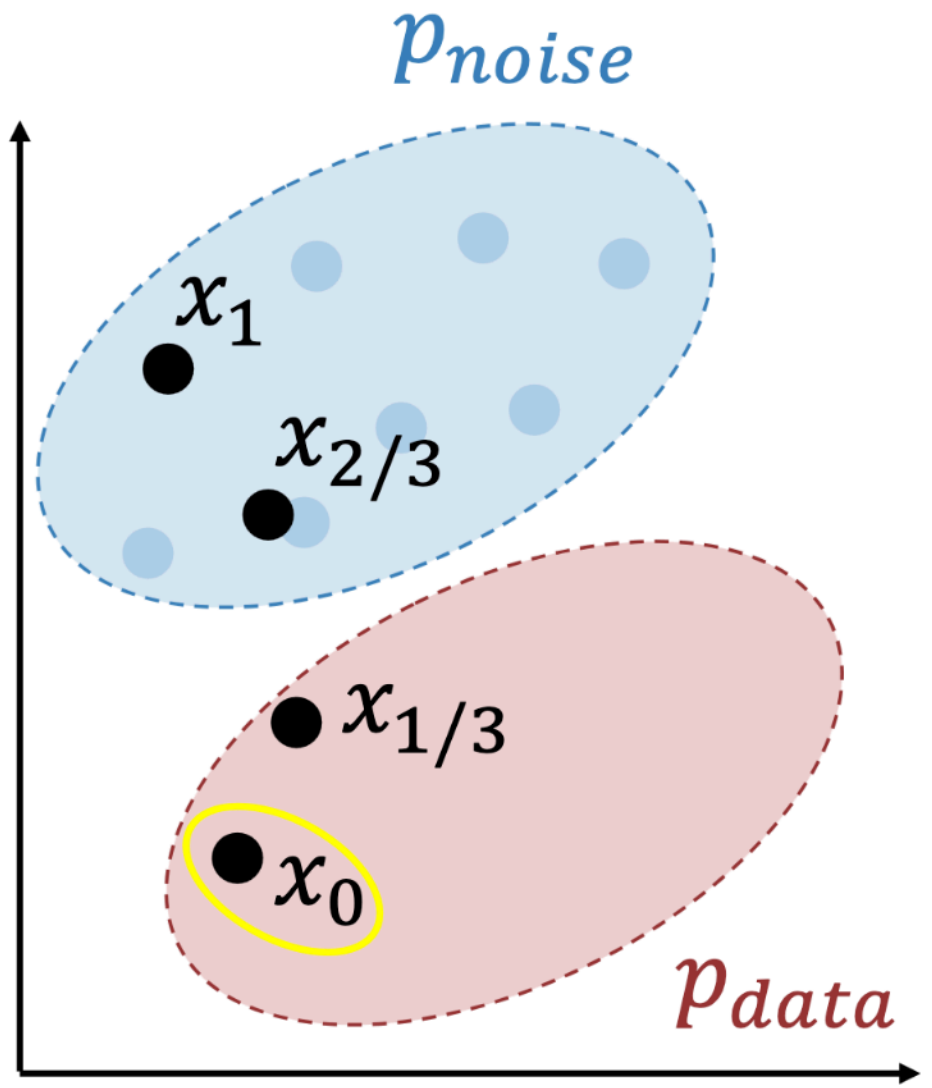
Sample $x \sim p_{noise}$

For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

  Evaluate $v_t = f_\theta(x_t, t)$

  Step $x = x - v_t/T$

Return x



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

Rectified Flow: Sampling

Choose number of steps T (often T=50)

Sample $x \sim p_{noise}$

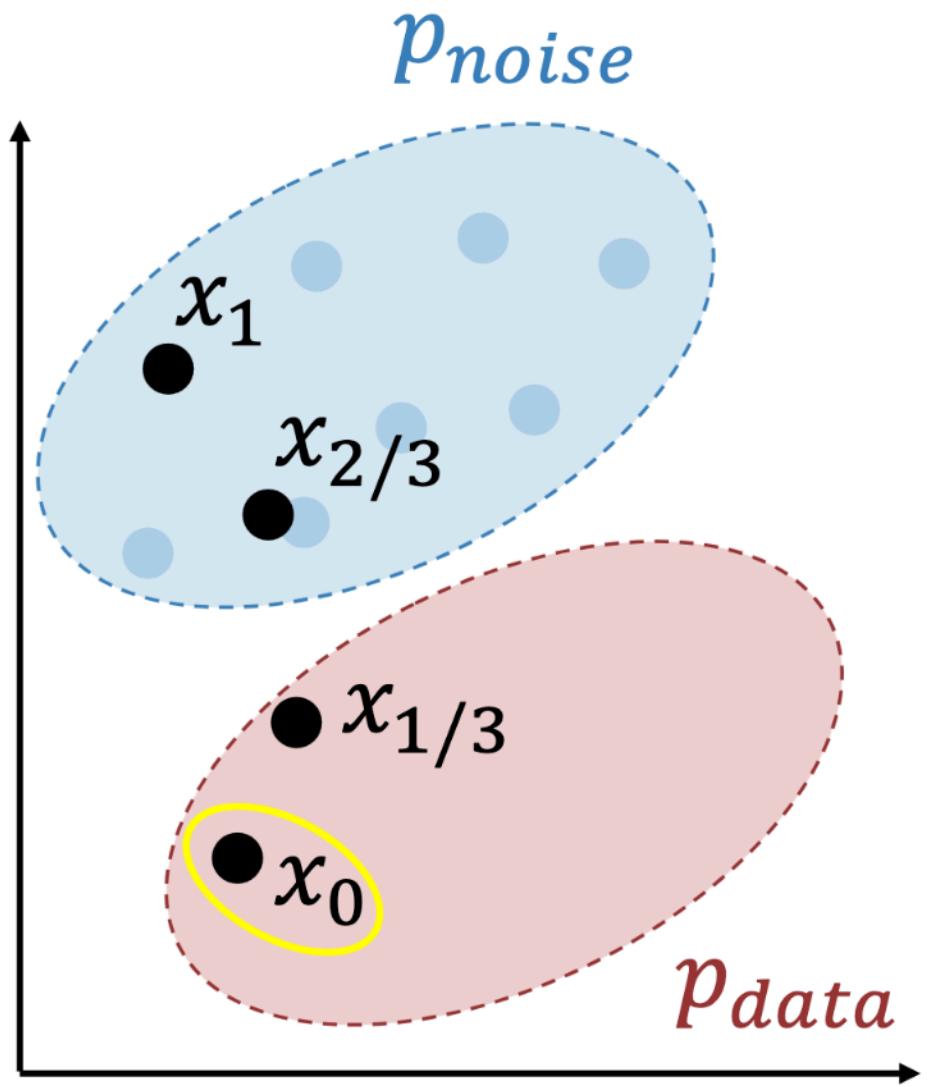
For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \ldots, 0]$:

  Evaluate $v_t = f_\theta(x_t, t)$

  Step $x = x - v_t/T$

Return x

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

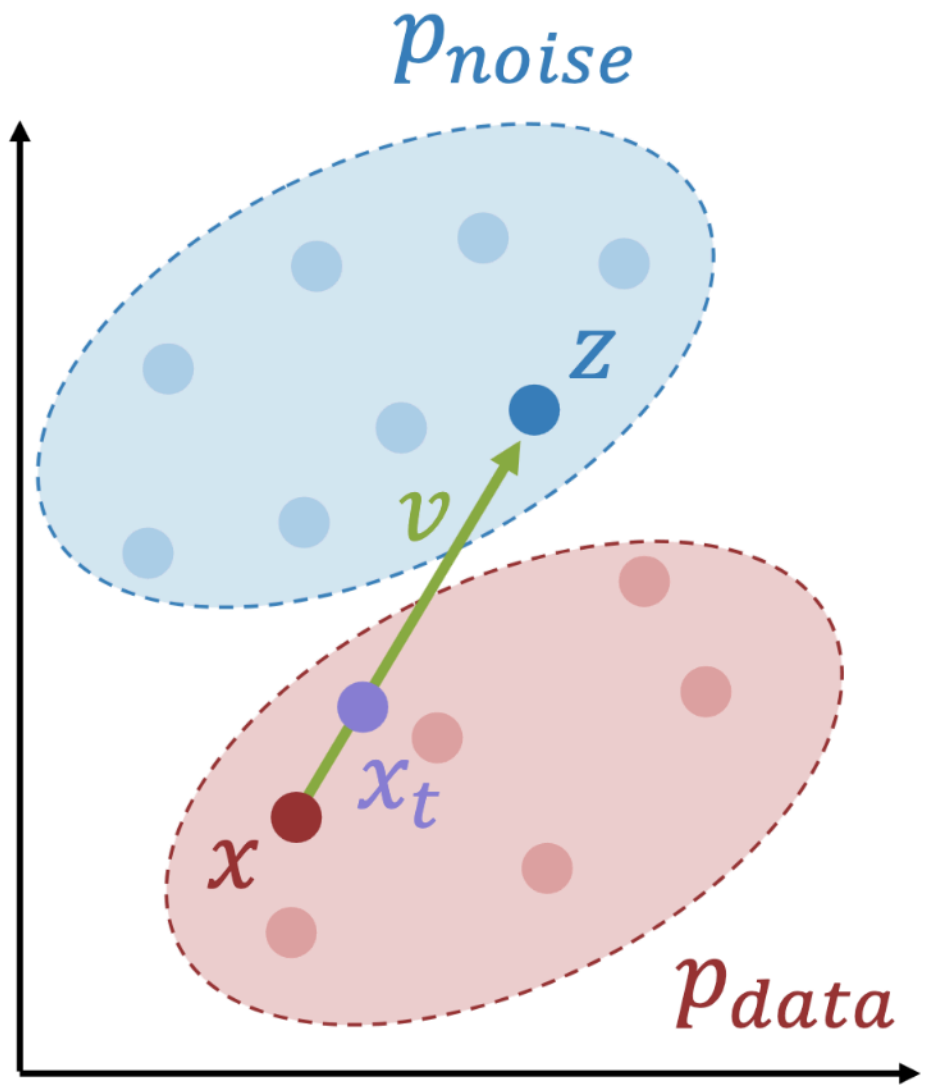
Rectified Flow: Summary

Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

What is “rectified“ about it?

Many different possible flows:

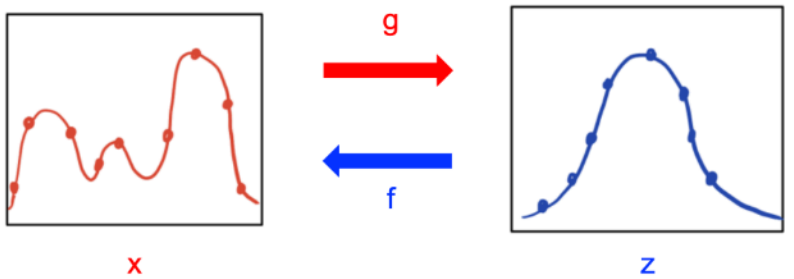
We introduce *rectified flow*, a surprisingly simple approach to the transport mapping problem, which unifiedly solves both generative modeling and domain transfer. The rectified flow is an ODE model that transport distribution π_0 to π_1 by *following straight line paths as much as possible.* The straight paths are preferred both theoretically because it is the shortest path between two end points, and computationally because it can be exactly simulated without time discretization. Hence, flows with straight paths bridge the gap between one-step and continuous-time models.

graphic by Fjedle et al, link

Rectified Flows are Continuous Normalizing Flows

maps between two distributions like a normalizing flow

$$x = F(z_0) = z_T = z_0 + \int_0^T f(z_t, t)dt$$



$$\int p_x(x)dx = \int p_z(g(x))dz$$

$$p_x(x) = p_z(g(x))\left|\frac{dg(x)}{dx}\right| \rightarrow p_z(g(x))|det.J(g(x))|$$

$$\log p_x(x) = \log p_z(g(x)) + \log|det.J(g(x))|$$

Rectified Flows are Continuous Normalizing Flows

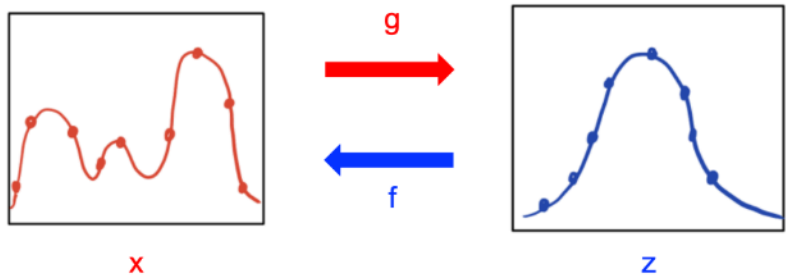
maps between two distributions like a normalizing flow

$$x = F(z_0) = z_T = z_0 + \int_0^T f(z_t, t)dt$$

so can I evaluate p(x)?

$$\log(p(x)) = \log(p(z_0)) - \int_0^T \mathrm{Tr}\left[\frac{\partial f}{\partial z_t}\right] dt$$

The Hutchinson trick is $\mathrm{tr}\, A = \mathbb{E}_\varepsilon[\varepsilon^t A \varepsilon]$ for any random variable $\varepsilon \in \mathbb{R}^d$ having centered, independent components of variance 1. In practice, the expectation is approximated by Monte-Carlo sampling, usually using only one realization of ε



$$\int p_x(x)dx = \int p_z(g(x))dz$$

$$p_x(x) = p_z(g(x))\left|\frac{dg(x)}{dx}\right| \rightarrow p_z(g(x))|det.J(g(x))|$$

$$\log p_x(x) = \log p_z(g(x)) + \log|det.J(g(x))|$$

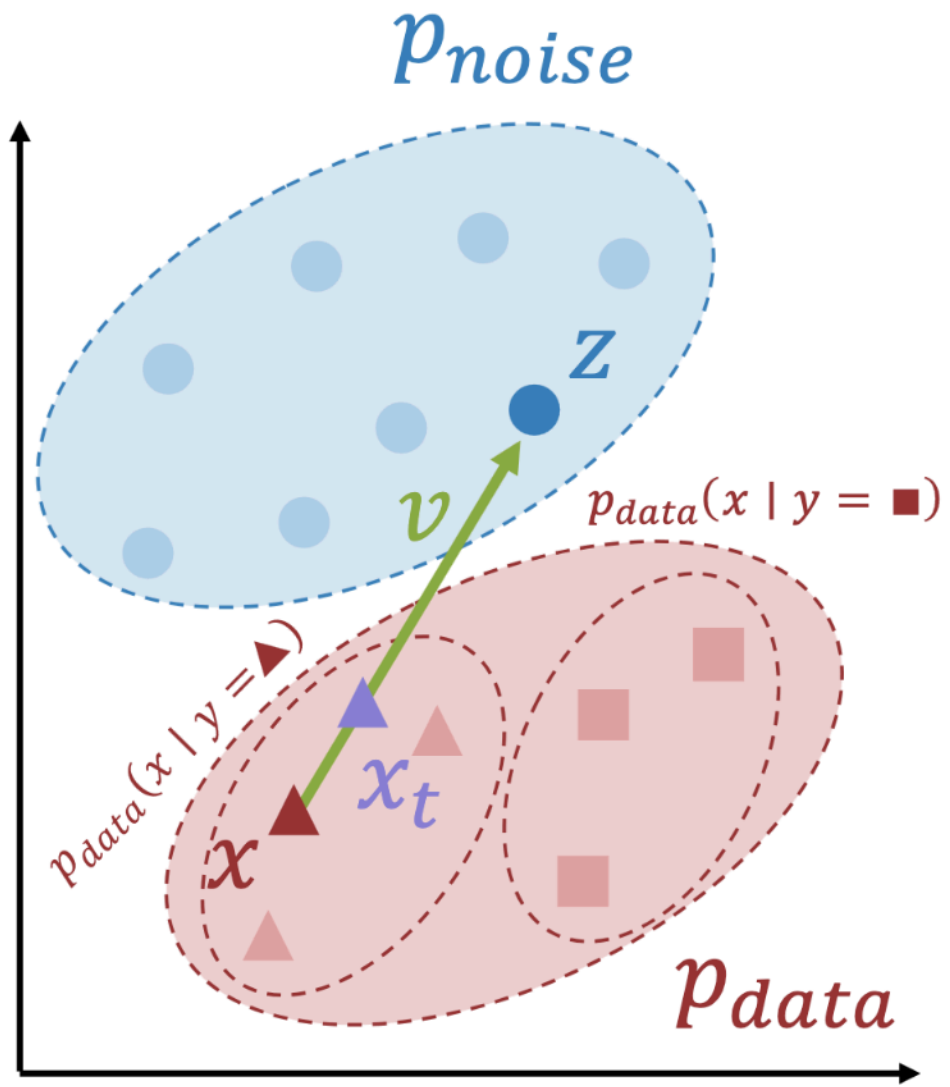
Conditional Rectified Flow

Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

slide by Li et al, Stanford CS231n

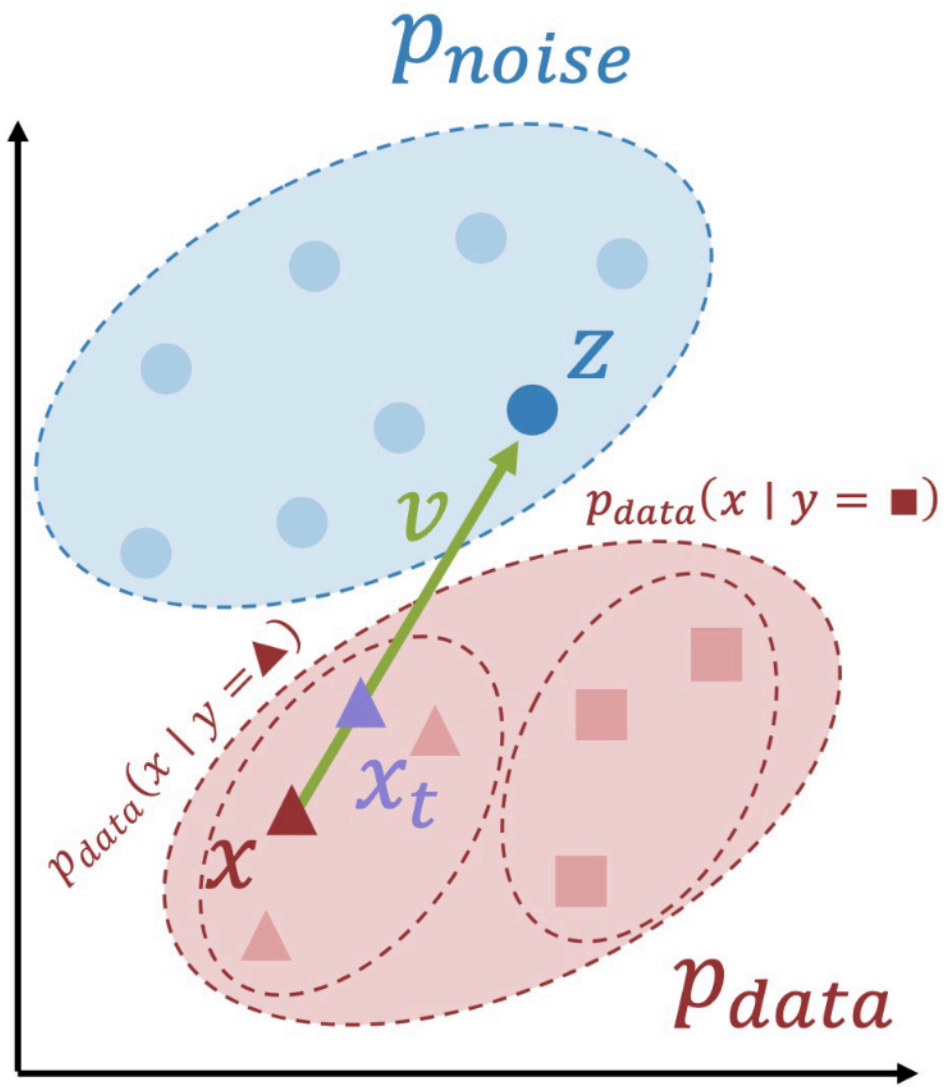
Conditional Rectified Flow

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

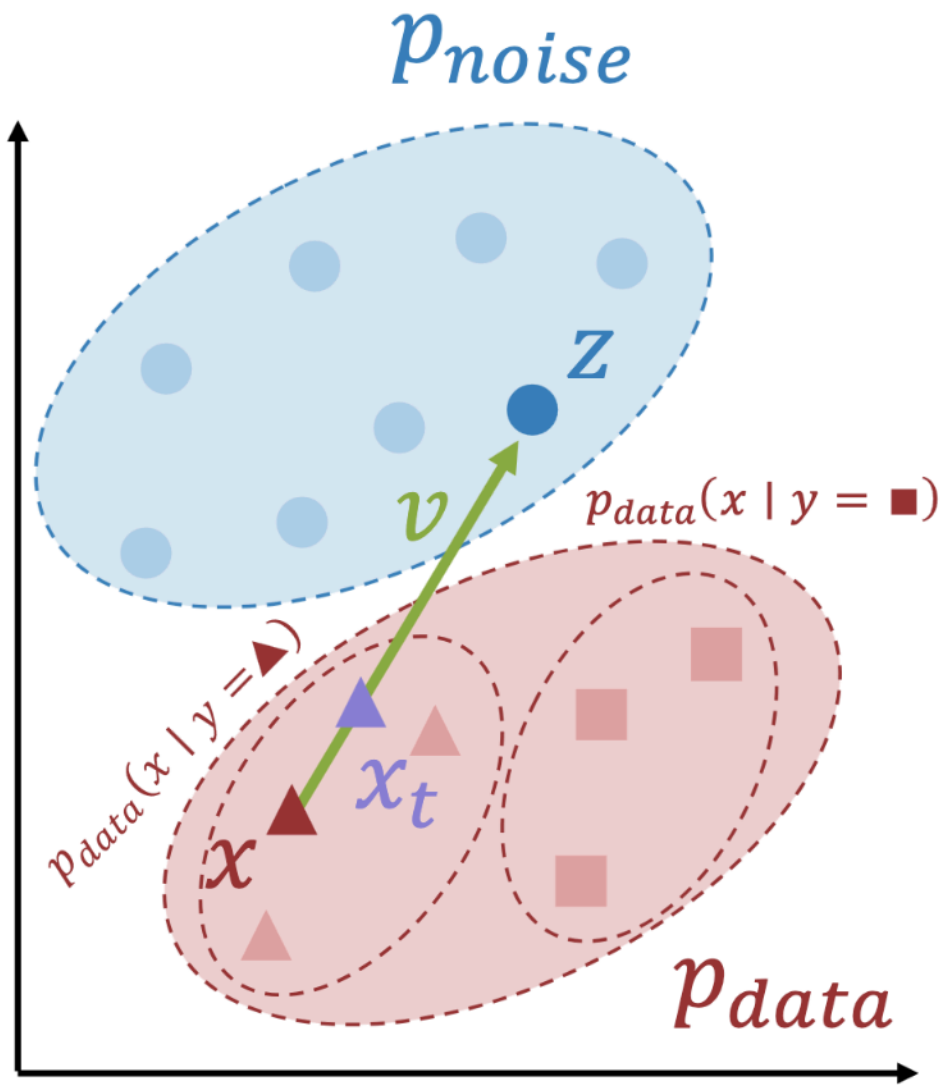
slide by Li et al, Stanford CS231n

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we "emphasize" the conditioning y?



Randomly drop y during training.

Now the same model is conditional and unconditional!

Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

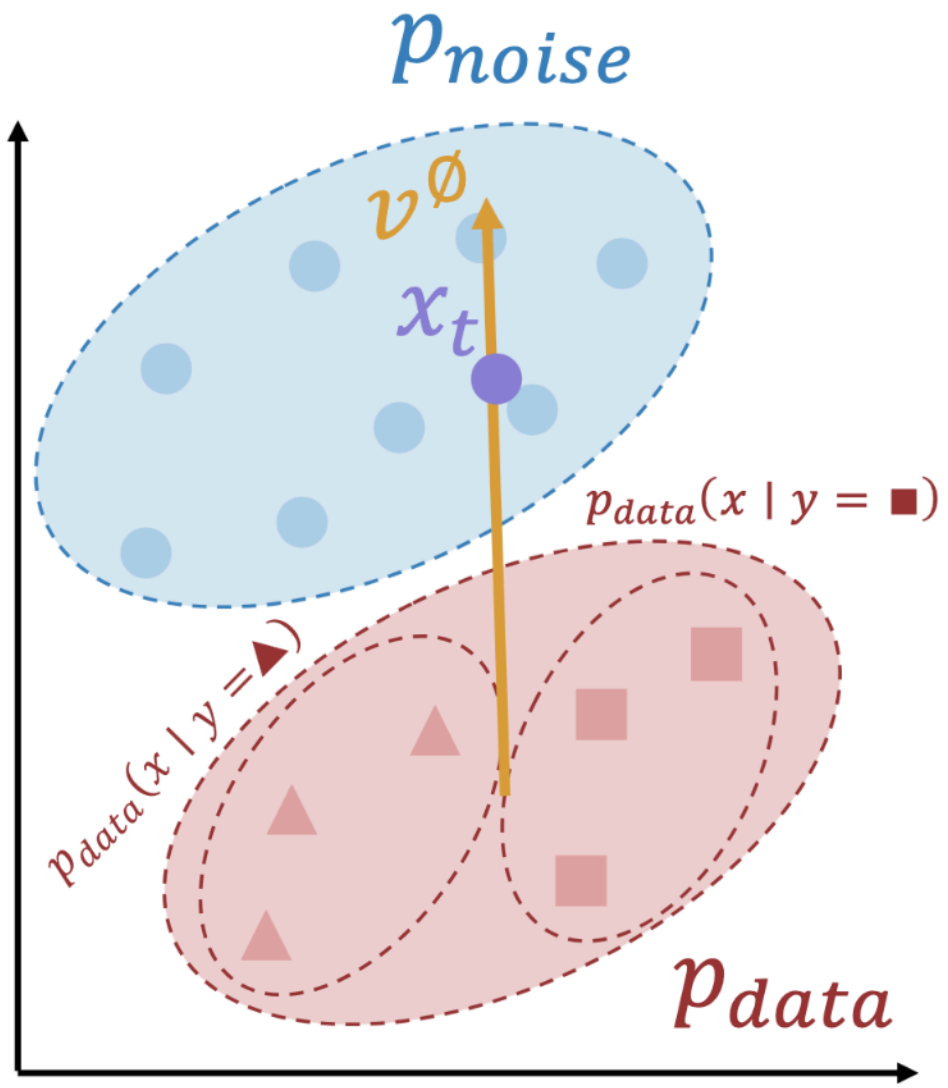
slide by Li et al, Stanford CS231n

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we "emphasize" the conditioning y?



Randomly drop y during training.

Now the same model is conditional and unconditional!
Consider a noisy x_t:
$v^{\emptyset} = f_\theta(x_t, y_\emptyset, t)$ points toward $p(x)$

Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

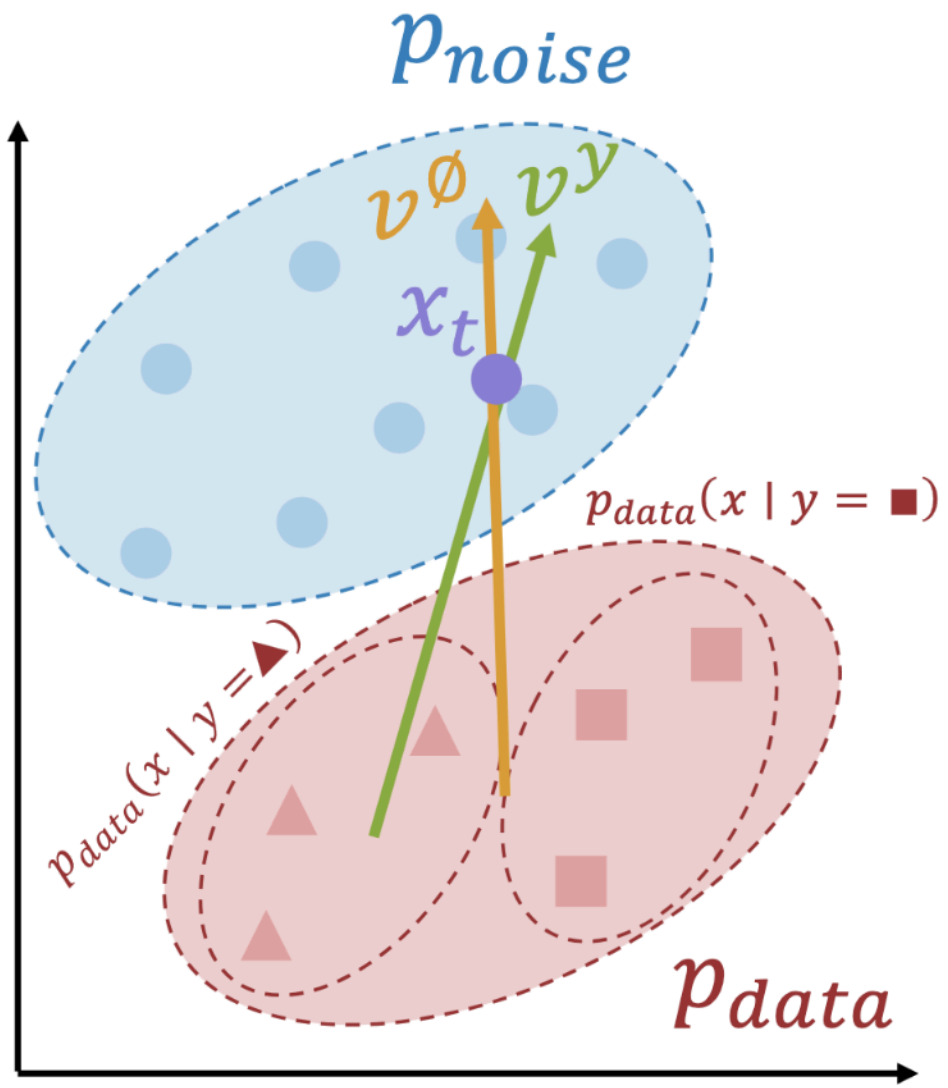
slide by Li et al, Stanford CS231n

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we "emphasize" the conditioning y?



Randomly drop y during training.

Now the same model is conditional and unconditional!
Consider a noisy x_t:
$v^{\emptyset} = f_\theta(x_t, y_\emptyset, t)$ points toward $p(x)$
$v^{y} = f_\theta(x_t, y, t)$ points toward $p(x \mid y)$

Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

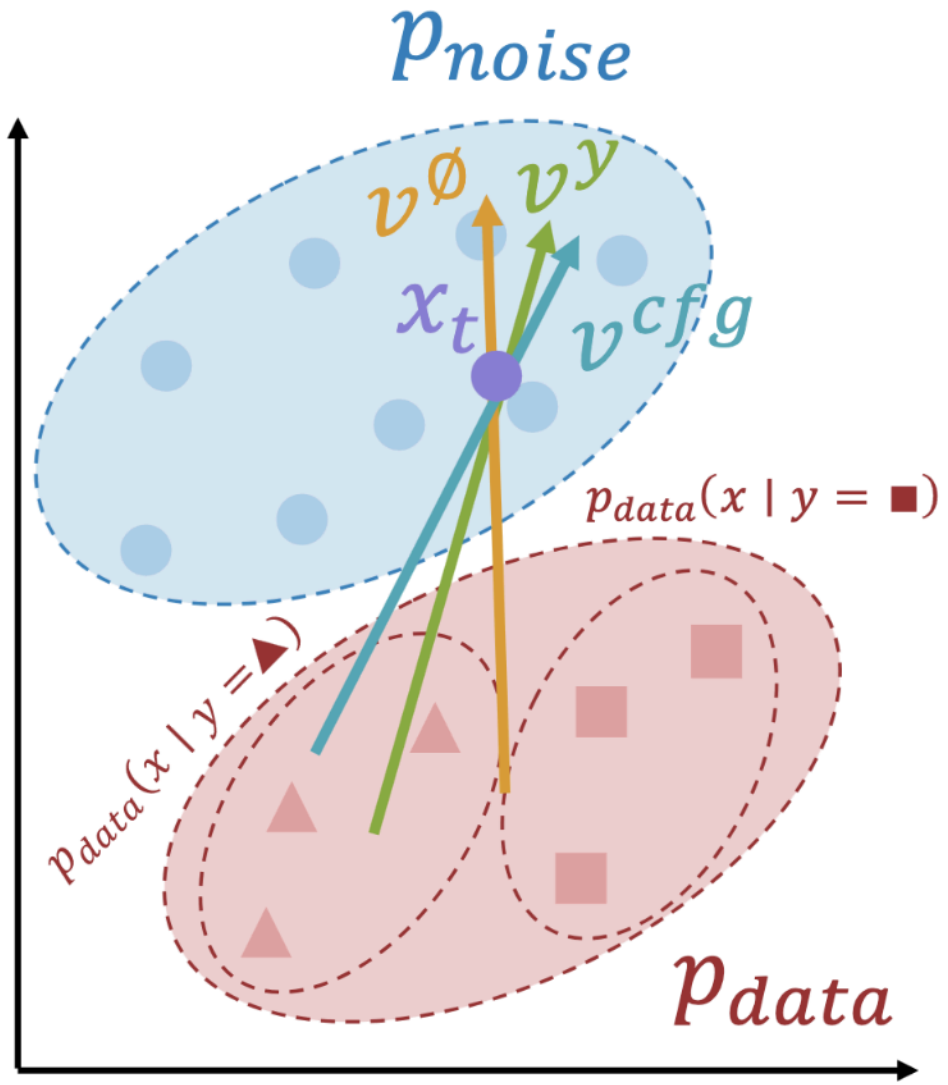
slide by Li et al, Stanford CS231n

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we "emphasize" the conditioning y?



Randomly drop y during training.

Now the same model is conditional and unconditional!
Consider a noisy x_t:

$v^{\emptyset} = f_\theta(x_t, y_\emptyset, t)$ points toward $p(x)$

$v^{y} = f_\theta(x_t, y, t)$ points toward $p(x \mid y)$

$v^{cfg} = (1+w)v^{y} - wv^{\emptyset}$ points more toward $p(x \mid y)$

Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

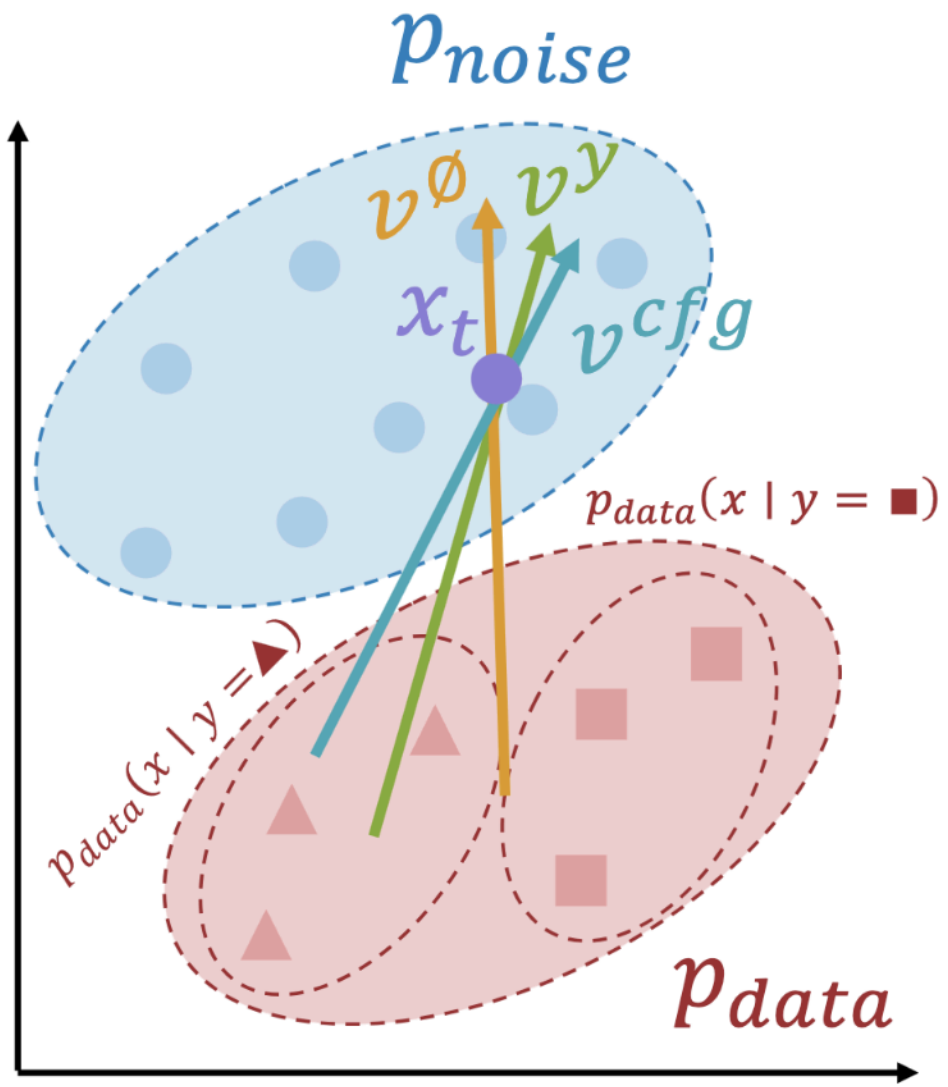
slide by Li et al, Stanford CS231n

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Can we control how much we "emphasize" the conditioning y?



Randomly drop y during training.

Now the same model is conditional and unconditional!

Consider a noisy x_t:

$v^{\emptyset} = f_\theta(x_t, y_\emptyset, t)$ points toward $p(x)$

$v^{y} = f_\theta(x_t, y, t)$ points toward $p(x \mid y)$

$v^{cfg} = (1+w)v^{y} - wv^{\emptyset}$ points more toward $p(x \mid y)$

During sampling, step according to v^{cfg}

Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

slide by Li et al, Stanford CS231n

Classifier-Free Guidance (CFG)

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

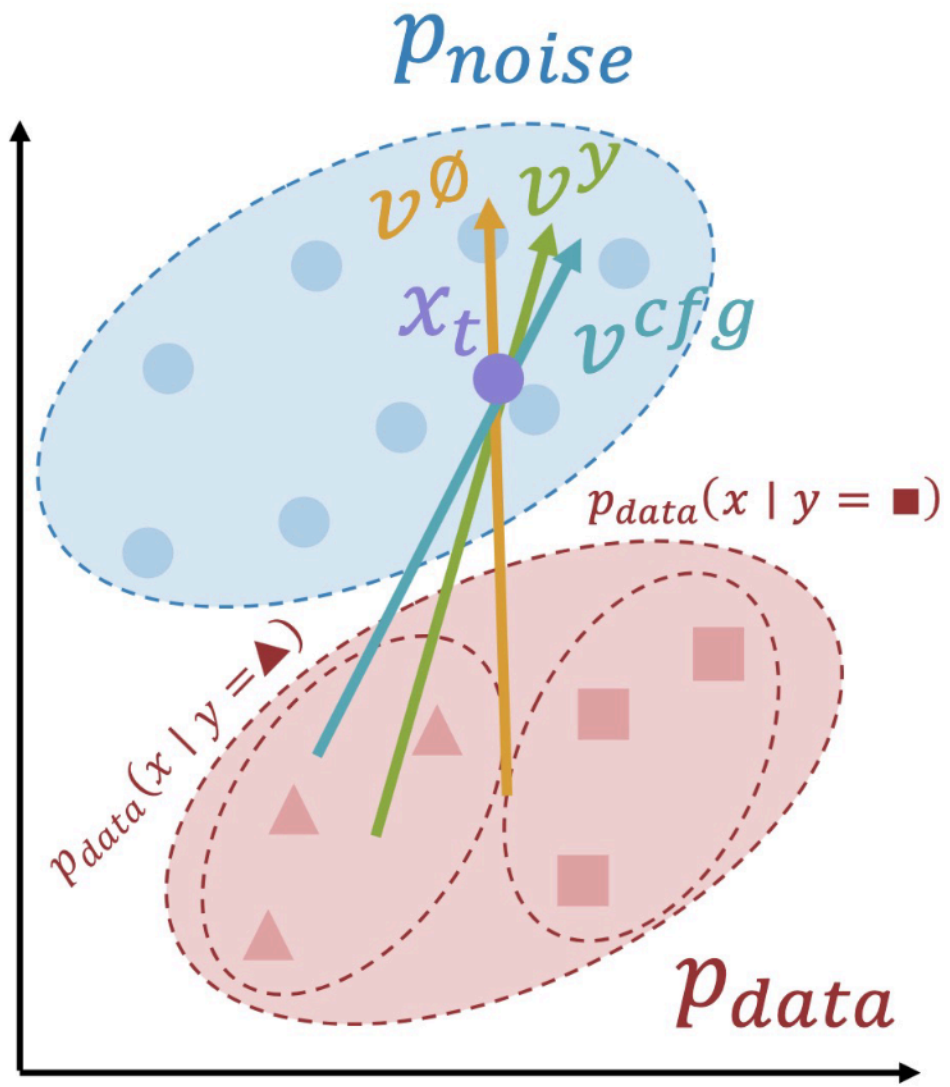
Can we control how much we "emphasize" the conditioning y?

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Used everywhere in practice! Very important for high-quality outputs

Doubles the cost of sampling…



Dhariwal and Nichol, "Diffusion Models beat GANs on Image Synthesis", arXiv 2021
Ho and Salimans, "Classifier-Free Diffusion Guidance", arXiv 2022

slide by Li et al, Stanford CS231n

Latent Diffusion

This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

High-Resolution Image Synthesis with Latent Diffusion Models

Robin Rombach[1] * Andreas Blattmann[1] * Dominik Lorenz[1] Patrick Esser[R] Björn Ommer[1]
[1]Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany [R]Runway ML

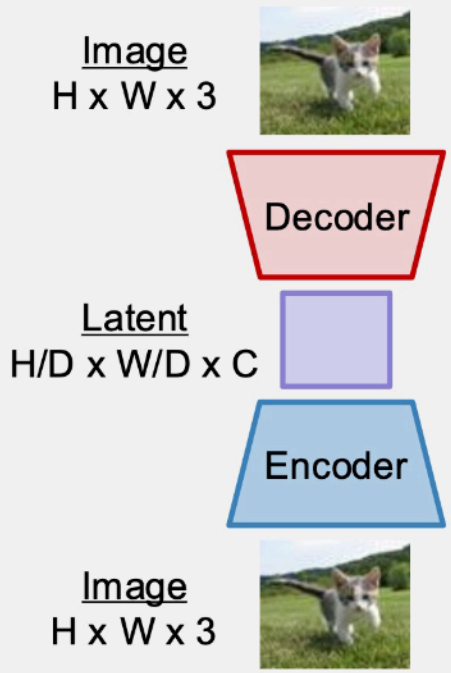
https://github.com/CompVis/latent-diffusion

Abstract

By decomposing the image formation process into a sequential application of denoising autoencoders, diffusion

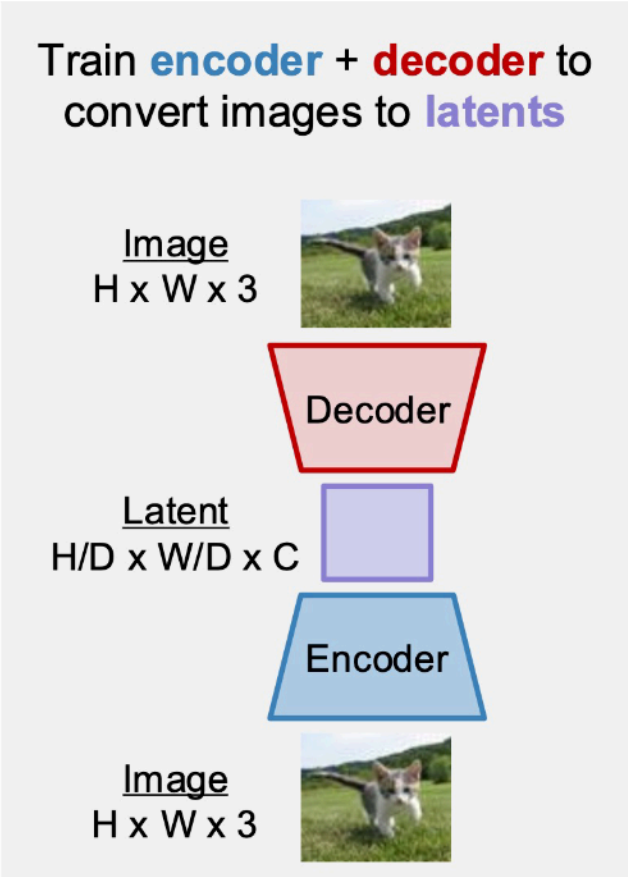
Latent Diffusion

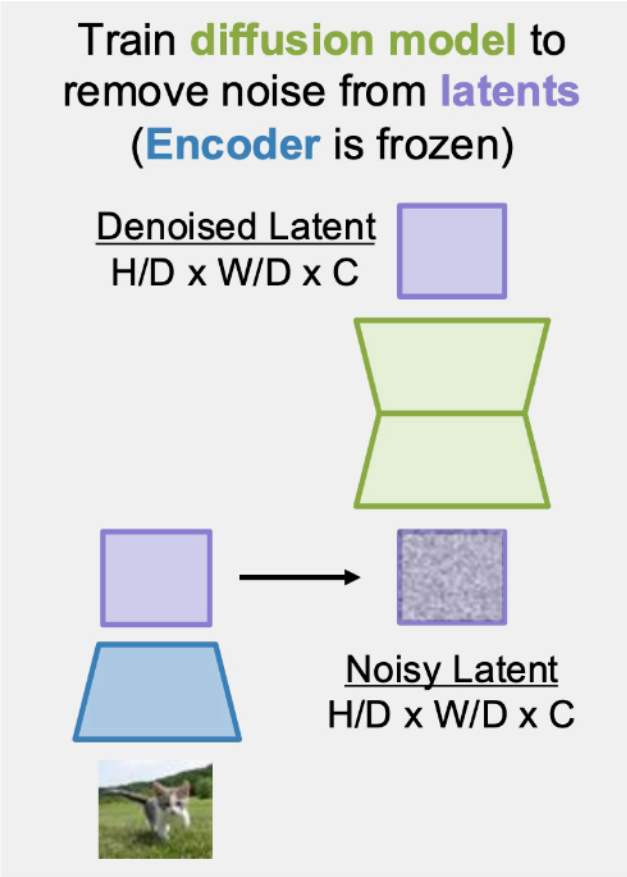
Train **encoder** + **decoder** to convert images to **latents**



slide by Li et al, Stanford CS231n

Latent Diffusion

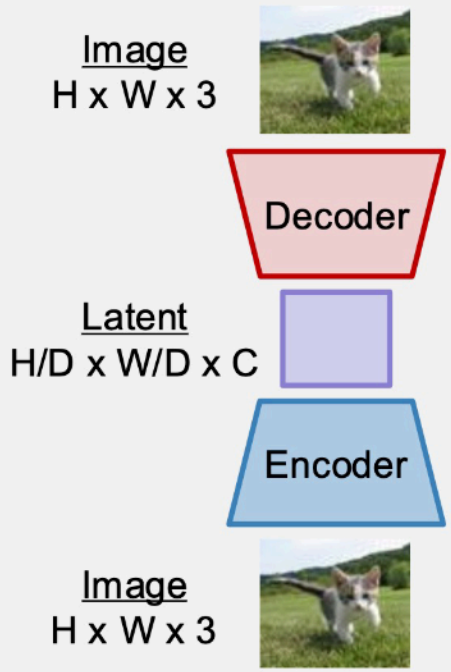




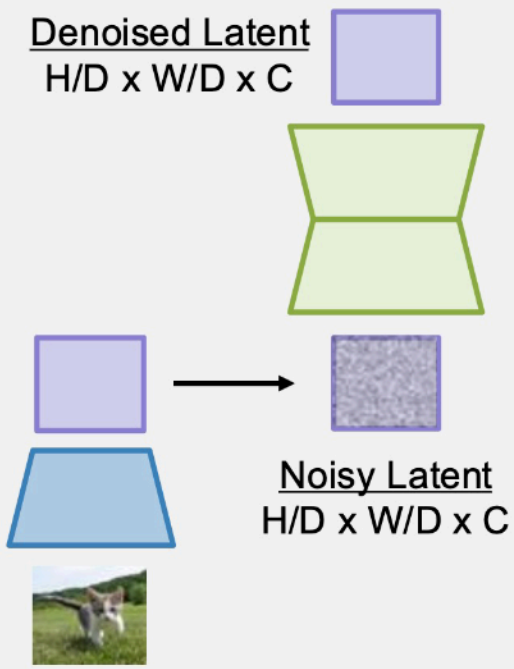
slide by Li et al, Stanford CS231n

Latent Diffusion

Train **encoder** + **decoder** to convert images to **latents**



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)



After training:

Sample random **latent**

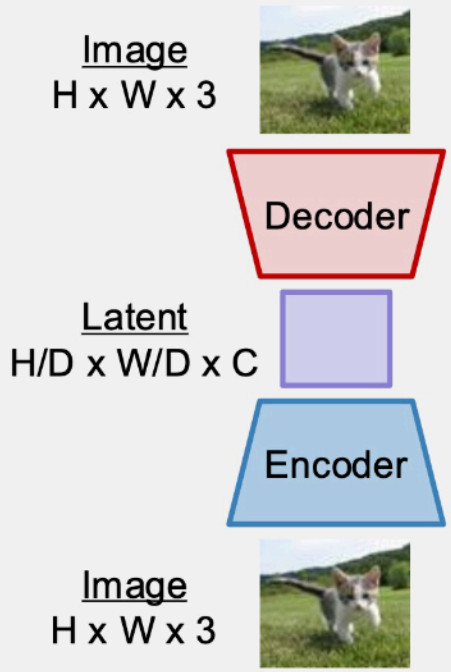
Iteratively apply **diffusion model** to remove noise

run **decoder** to get **image**

slide by Li et al, Stanford CS231n

Latent Diffusion

Train **encoder** + **decoder** to convert images to **latents**

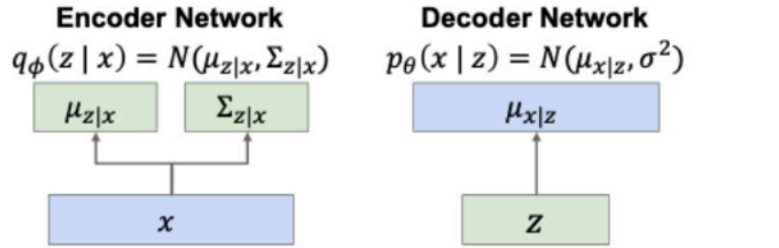


How do we train the encoder+decoder?

Solution: It's a VAE! Typically with very small KL prior weight

Recall: VAE

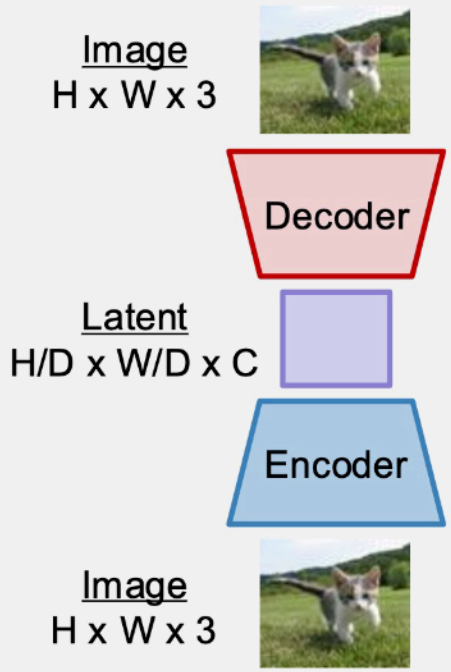
$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$



slide by Li et al, Stanford CS231n

Latent Diffusion

Train **encoder** + **decoder** to convert images to **latents**



How do we train the encoder+decoder?

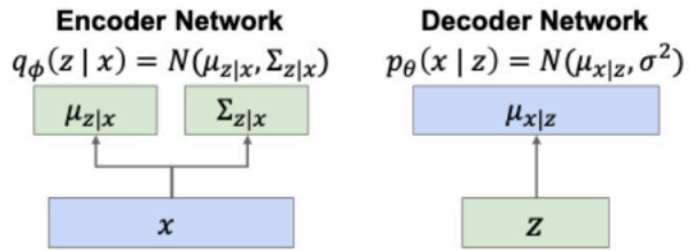
Solution: It's a VAE! Typically with very small KL prior weight

Problem: Decoder outputs often blurry

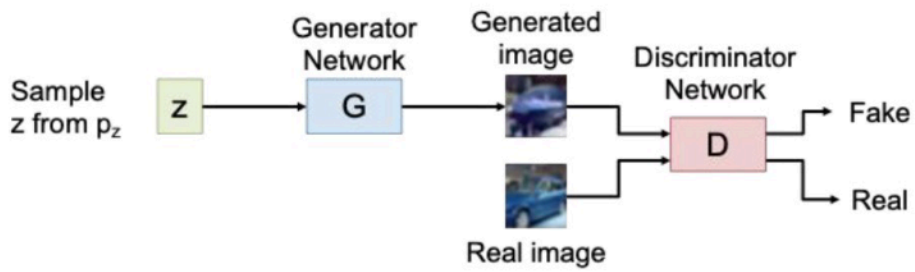
Solution: Add a discriminator!

Recall: VAE

$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$

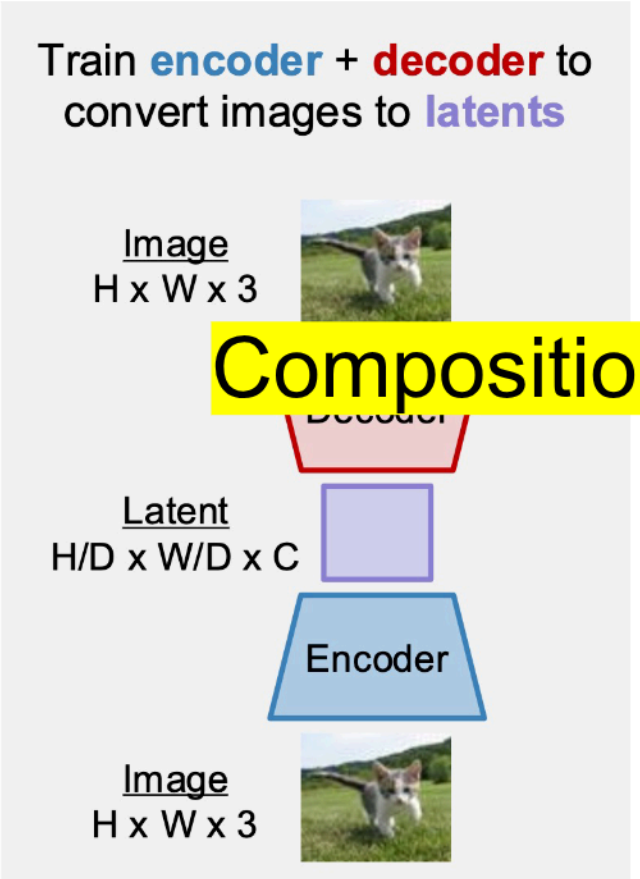


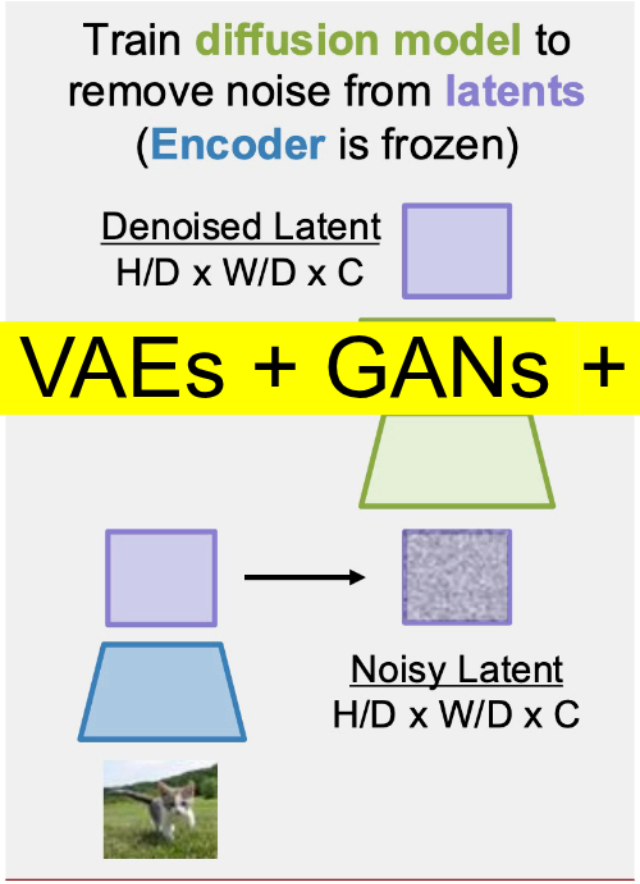
Recall: GAN

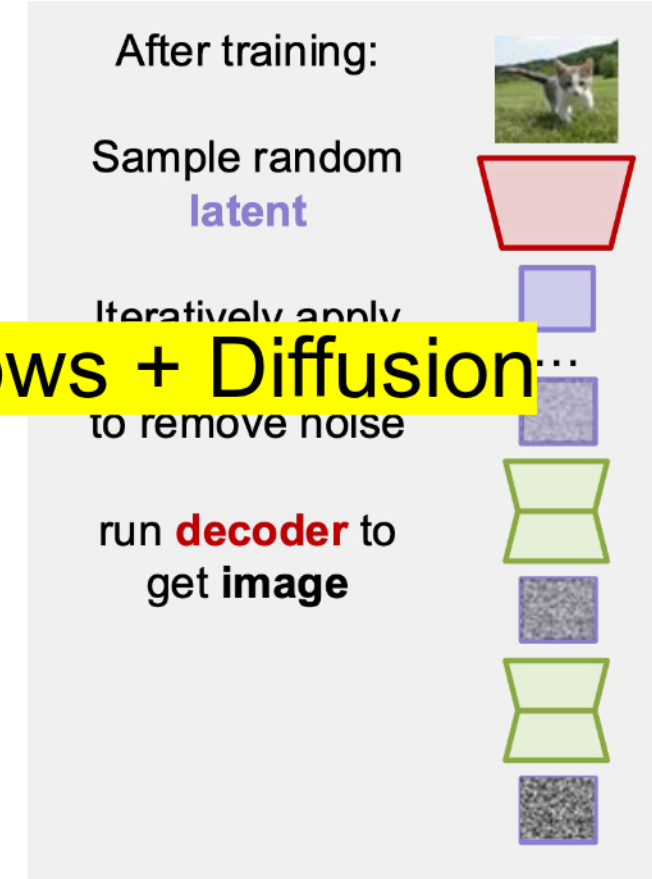


slide by Li et al, Stanford CS231n

Latent Diffusion







Composition of VAEs + GANs + Flows + Diffusion

slide by Li et al, Stanford CS231n

Autoregressive Models

slide by Li et al, Stanford CS231n

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

slide by Li et al, Stanford CS231n

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \ldots x^{(N)}$, train the model by solving:

$$W^* = \arg\max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

slide by Li et al, Stanford CS231n

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \ldots x^{(N)}$, train the model by solving:

$$W^* = \arg\max_W \prod_i p(x^{(i)})$$

Maximize probability of training data
(Maximum likelihood estimation)

$$= \arg\max_W \sum_i \log p(x^{(i)})$$

Log trick: Swap product and sum

$$= \arg\max_W \sum_i \log f(x^{(i)}, W)$$

This is our loss function.
maximize it with gradient descent

slide by Li et al, Stanford CS231n

Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x is a sequence: $x = (x_1, x_2, \ldots, x_T)$

Use the chain rule of probability:

$$\begin{aligned} p(x) &= p(x_1, x_2, x_3, \ldots, x_T) \\ &= p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \ldots \\ &= \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}) \end{aligned}$$

slide by Li et al, Stanford CS231n

How can this be done for non-sequential data, e.g. images?

Autoregressive Models of Images

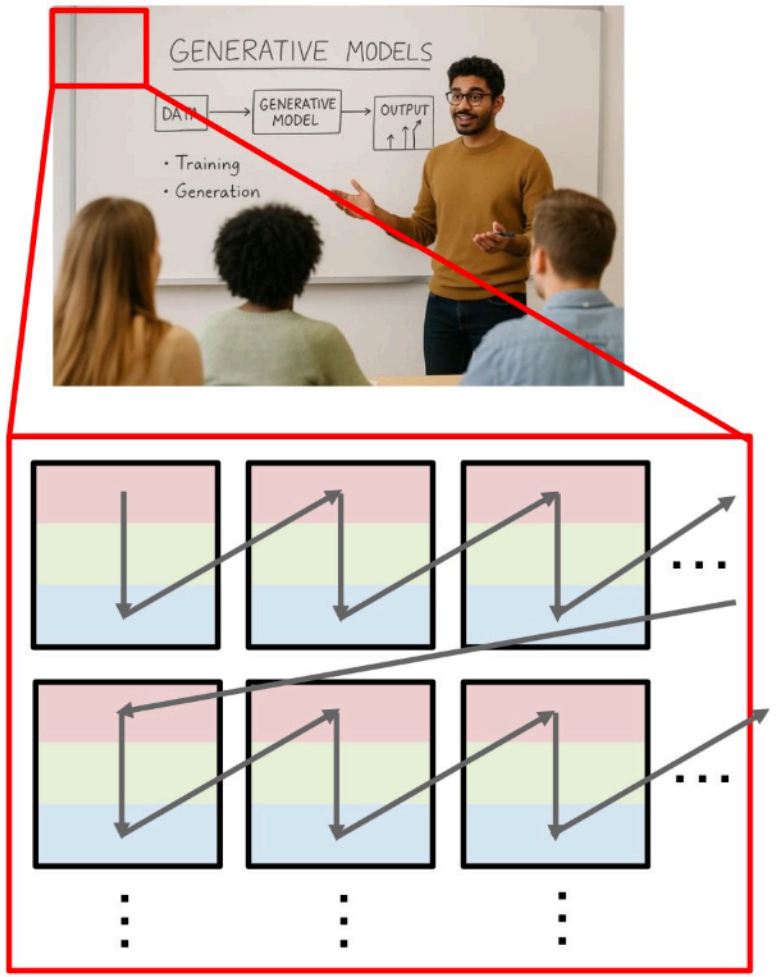
Treat an image as a sequence of 8-bit subpixel values (scanline order)

Predict each subpixel as a classification among 256 values [0…255]

Model with an RNN or Transformer

Problem: Too expensive. 1024x1024 image is a sequence of 3M subpixels

Solution (jumping ahead): Model as sequence of tiles, not sequence of subpixels



Diffusion Features

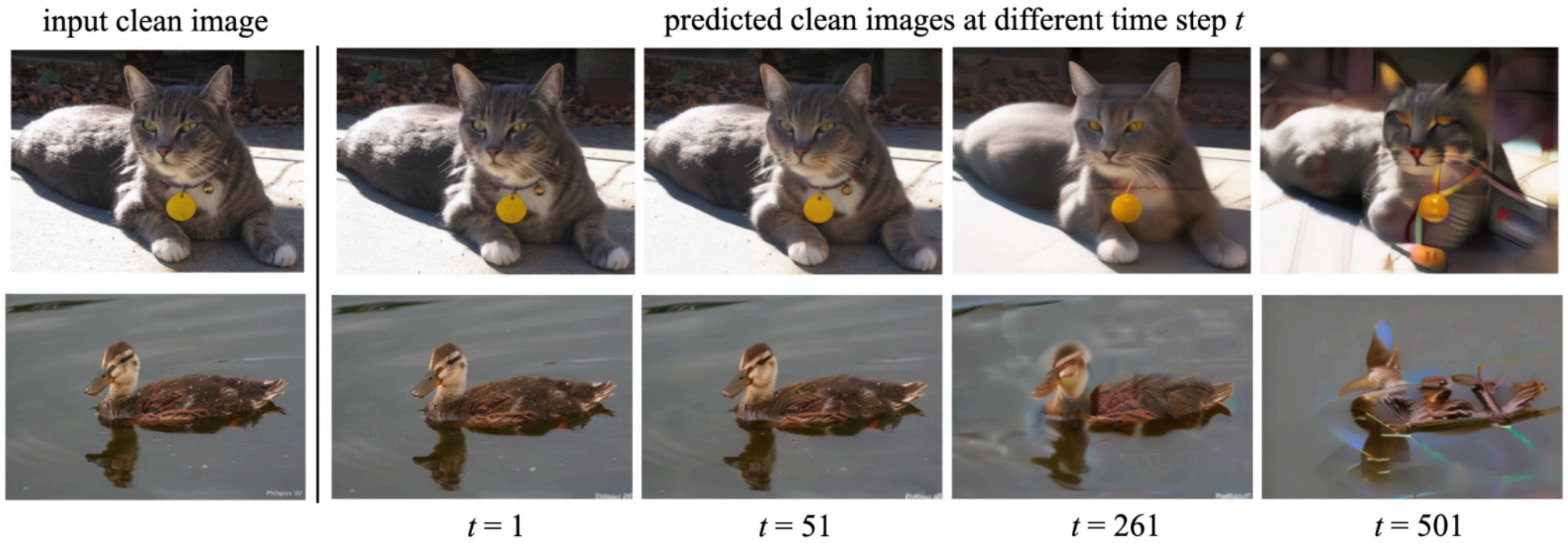
Emergent Correspondence from Image Diffusion

Luming Tang* **Menglin Jia*** **Qianqian Wang***
Cheng Perng Phoo **Bharath Hariharan**
Cornell University

Abstract

Finding correspondences between images is a fundamental problem in computer vision. In this paper, we show that correspondence emerges in image diffusion models *without any explicit supervision*. We propose a simple strategy to extract this implicit knowledge out of diffusion networks as image features, namely DIffusion FeaTures (DIFT), and use them to establish correspondences between real images. Without any additional fine-tuning or supervision on the task-specific data or annotations, DIFT is able to outperform both weakly-supervised methods and competitive off-the-shelf features in identifying semantic, geometric, and temporal

Diffusion Features

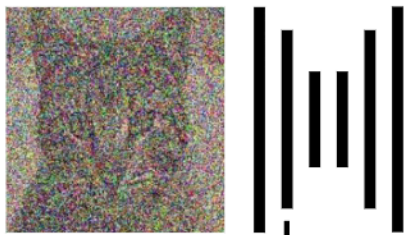


- observation: diffusion seems to follow abstraction hierarchies
- question: is this reflected in the network‘s features?

Diffusion Features

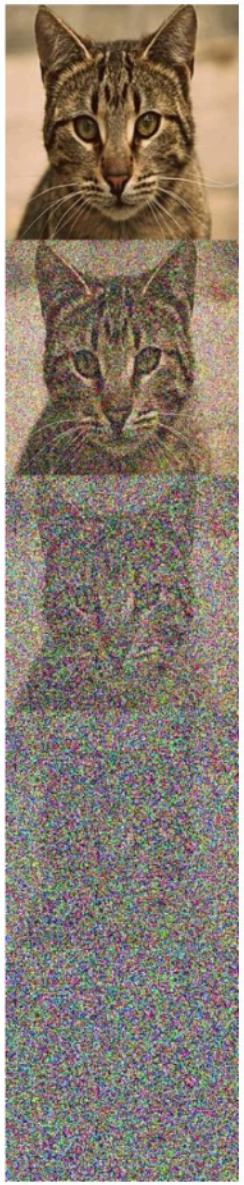
```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```

commonly, diffusion models are U-Nets:



$\varepsilon_t \longrightarrow z_t = z_{t+1} - \varepsilon_t$

1. For a given t, extract intermediate features from the U-Net
2. bilinear interpolation to pixel-level features
3. find pixel correspondances by cosine similarity



t = 0
No noise

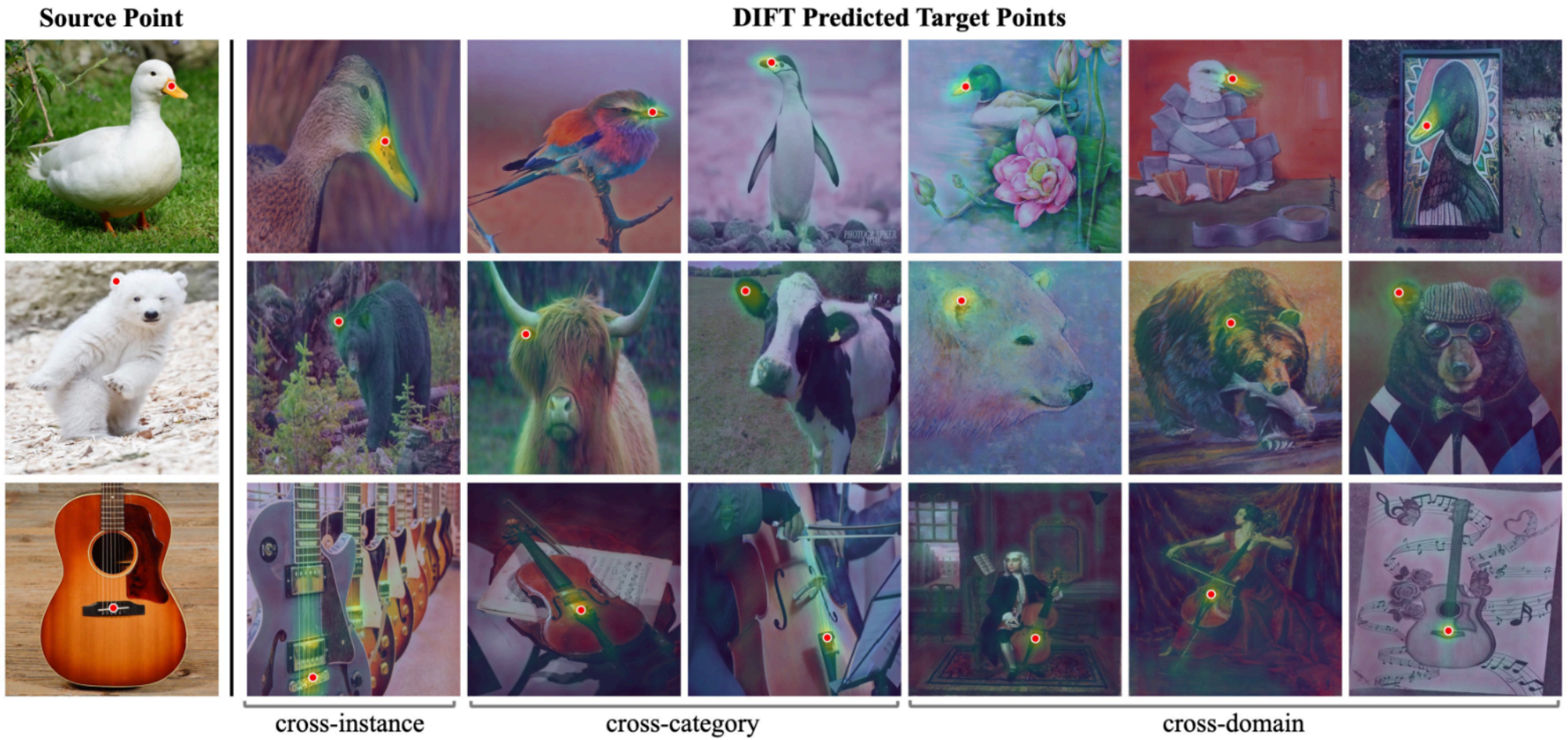
t = 1
Full noise

Diffusion Features are useful as self-supervised semantic correspondances.

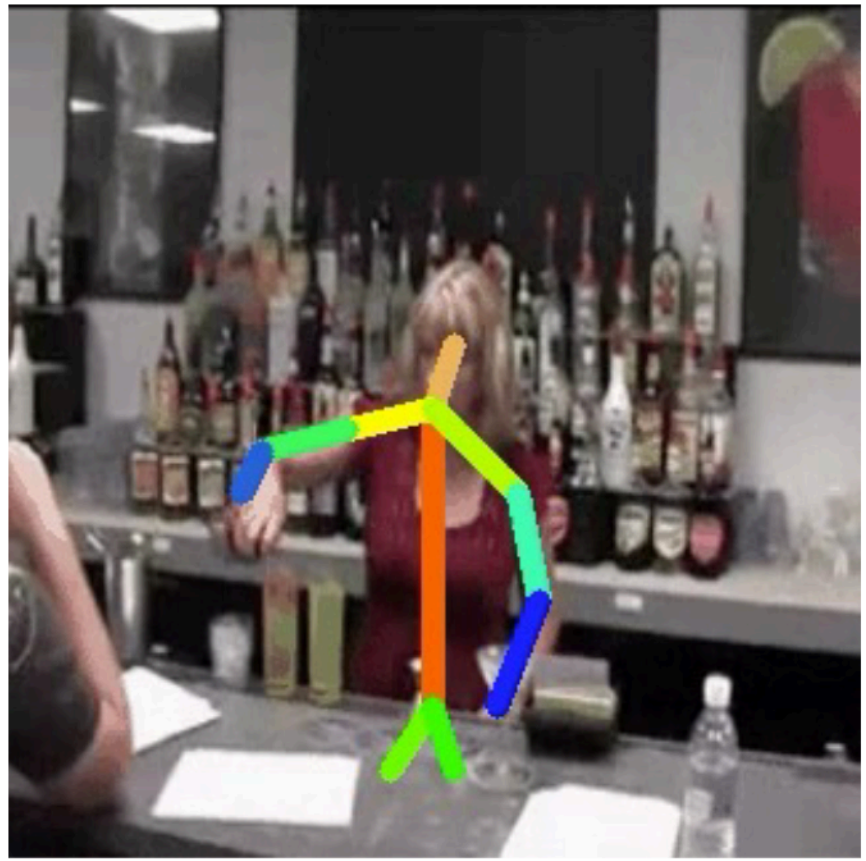
source image

target image

Diffusion Features are useful as self-supervised semantic correspondances.



Downstream Applications



Diffusion Policies

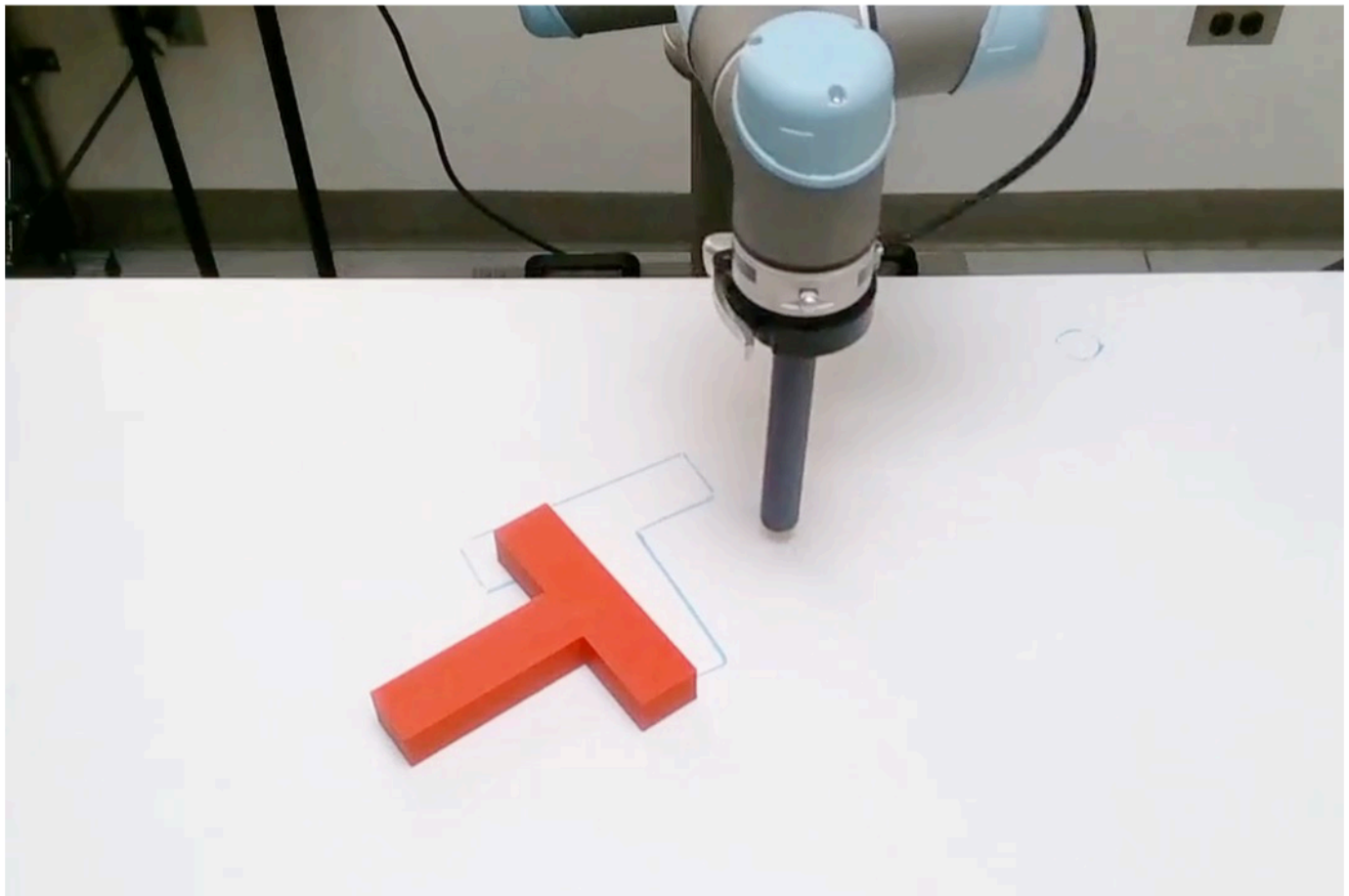
Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

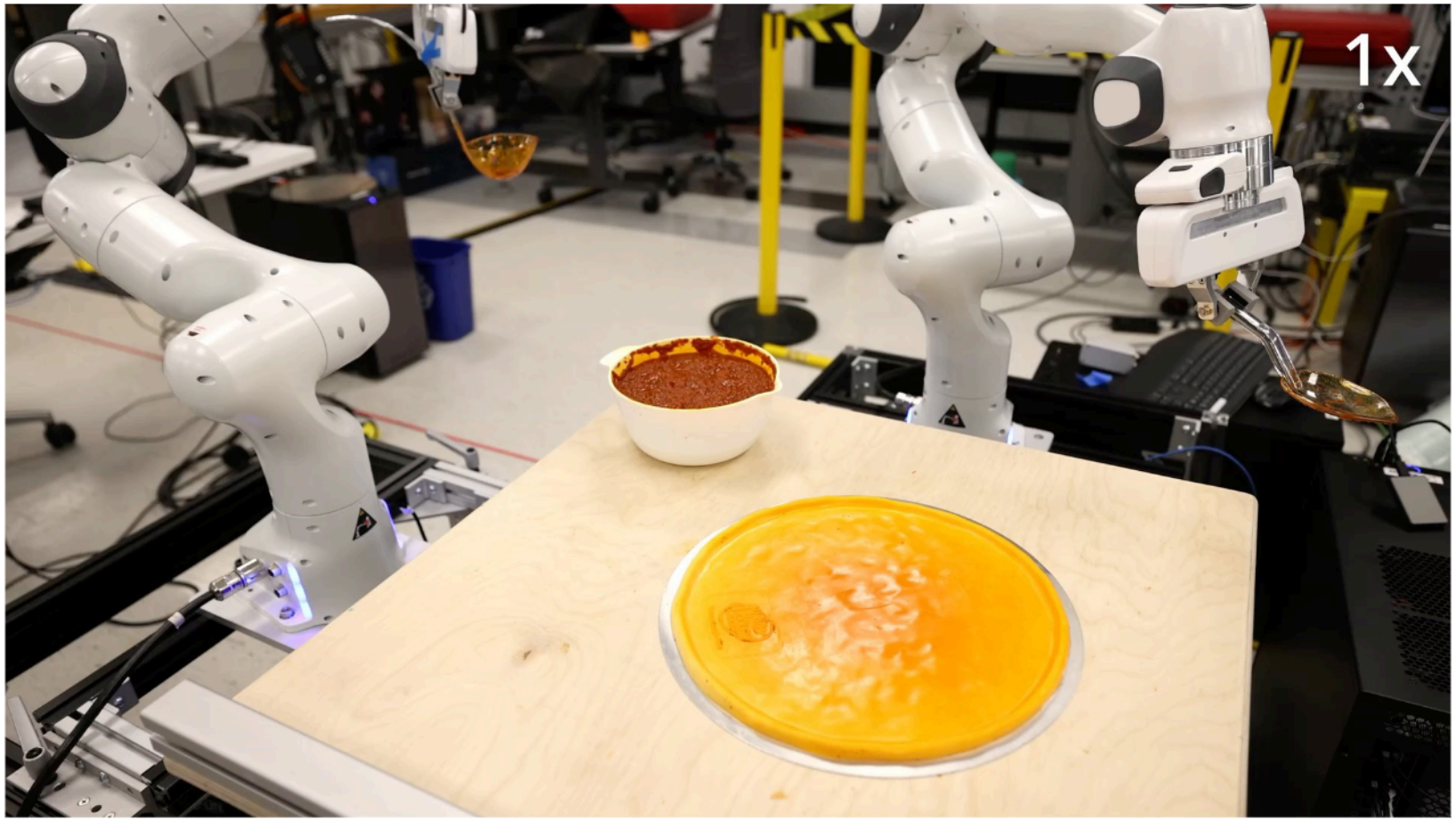
Cheng Chi*[1], Zhenjia Xu*[1], Siyuan Feng[2], Eric Cousineau[2], Yilun Du[3], Benjamin Burchfiel[2], Russ Tedrake [2,3], Shuran Song[1,4]

Abstract
This paper introduces Diffusion Policy, a new way of generating robot behavior by representing a robot's visuomotor policy as a conditional denoising diffusion process. We benchmark Diffusion Policy across 15 different tasks from 4 different robot manipulation benchmarks and find that it consistently outperforms existing state-of-the-art robot learning methods with an average improvement of 46.9%. Diffusion Policy learns the gradient of the action-distribution score function and iteratively optimizes with respect to this gradient field during inference via a series of stochastic Langevin dynamics steps. We find that the diffusion formulation yields powerful advantages when used for robot policies, including gracefully handling multimodal action distributions, being suitable for high-dimensional action spaces, and exhibiting impressive training stability. To fully unlock the potential of diffusion models for visuomotor policy learning on physical robots, this paper presents a set of key technical contributions including the incorporation of receding horizon control, visual conditioning, and the time-series diffusion transformer. We hope this work will help motivate a new generation of policy learning techniques that are able to leverage the powerful generative modeling capabilities of diffusion models. Code, data, and training details is available diffusion-policy.cs.columbia.edu

14 Mar 2024

Different Domain, also Multimodal Data: Diffusion of Robot Poses

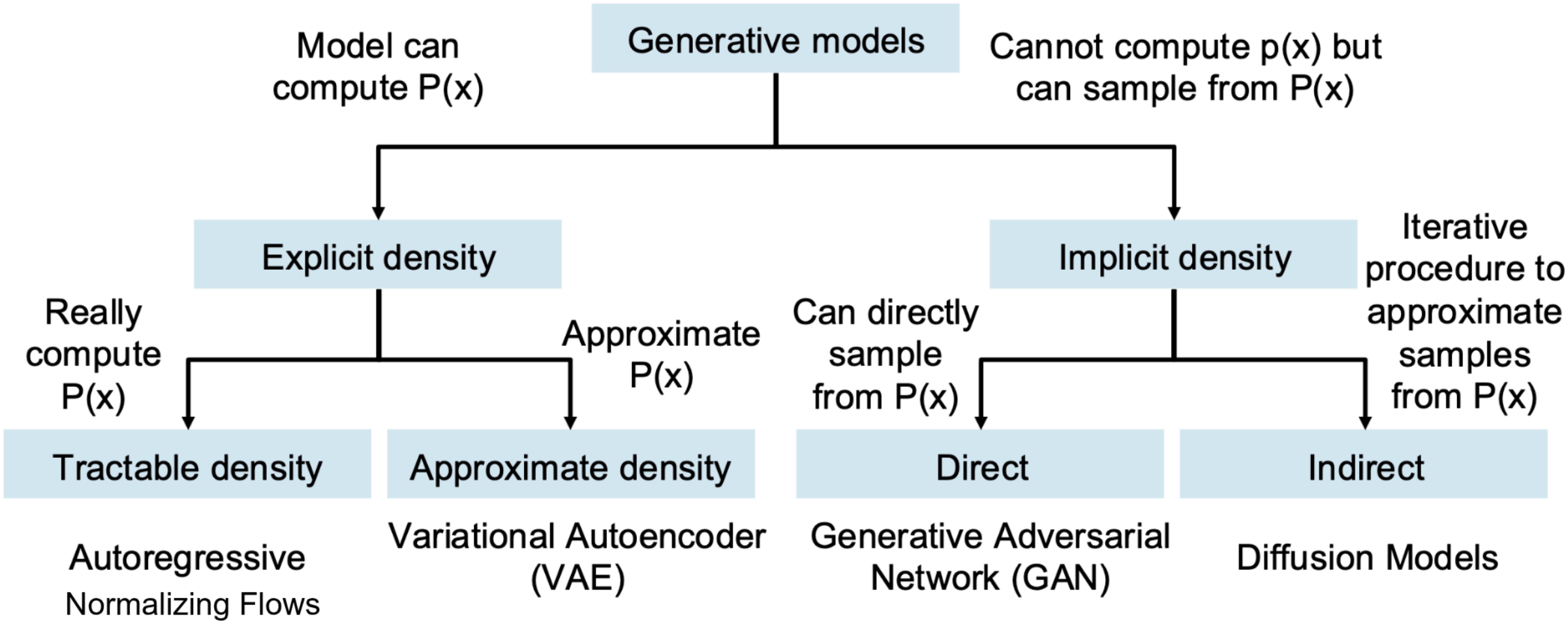




⚠ This is supervised imitation learning, not SSL.

Taxonomy of Generative Models

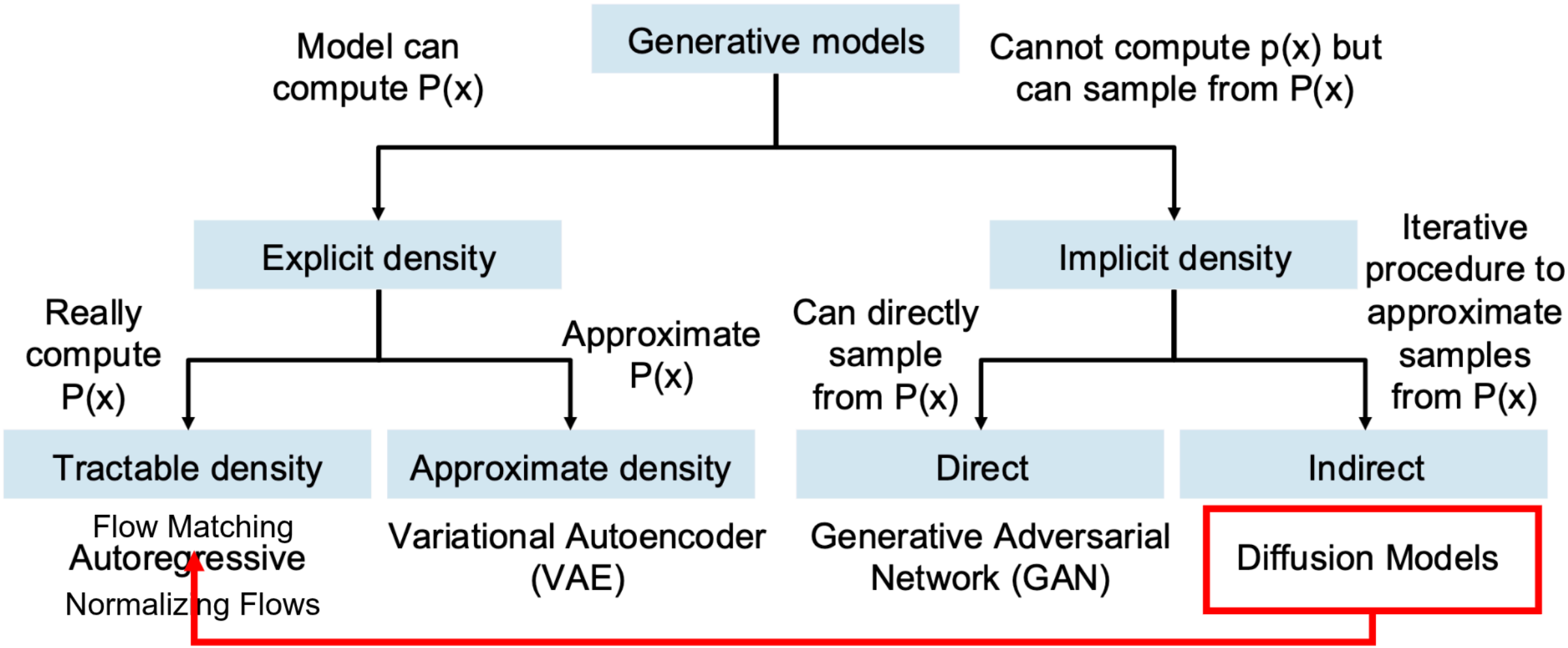
Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017



slide by Li et al, Stanford CS231n

Taxonomy of Generative Models

Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017



slide by Li et al, Stanford CS231n